

AXIS: Advanced eXpandable Iterative Stream-based Hash

Danping Shi, Lei Hu, Dingfeng Ye, Qianqian Yang, Qing Ling, Haomeng Xu,
Jun Xu, Guifang Huang, Yijing Wang, Zhiru Chen, Wenfei Qian, Xiaolong
Zhang, Pingjian Wang, and Yuan Ma

State Key Laboratory of Cyberspace Security Defense, Institute of Information
Engineering, Chinese Academy of Sciences, Beijing, China

Contents

1	GLOSSARY	4
1.1	Terms and Acronyms	4
1.2	Notations and Conventions	4
2	Specification of AXIS Hash	6
2.1	State	6
2.2	Mappings	8
2.3	Hashing	10
3	Security Cryptanalysis	11
3.1	Security Goals	11
3.2	Collision Attack	12
3.3	Preimage Attack	16
3.4	Length Extension Attack	17
3.5	Quantum Attack	17
4	Design Rationale	18
5	Feature Statement	19
6	Implementation and Test Vectors	20
6.1	Implementation Code	20
6.2	Known-Answer Test Vectors	21
7	Software Performance Evaluation	22
7.1	Test Platform	22
7.2	Reference Implementation Comparison	23
7.3	Optimized Implementation Comparison	25
7.4	Analysis and Conclusions	27
8	Randomness Test Results for Algorithm Submission	28
8.1	Test Results for Output Length 512 and Security Strength 256 ..	28
8.1.1	GM/T 0005-2021 Test Results	28
8.1.2	NIST SP 800-22 Test Results	29
8.2	Test Results for Output Length 768 and Security Strength 384 ..	37
8.2.1	GM/T 0005-2021 Test Results	38
8.2.2	NIST SP 800-22 Test Results	39
8.2.3	GM/T 0005-2021 Test Results	47
8.2.4	NIST SP 800-22 Test Results	48

List of Figures

1	The structure of AXIS	7
2	The update of a register	9

List of Tables

1	The constants of AXIS Hash	10
2	Security goals of AXIS Hash	12
3	Experimental estimation of algebraic degrees before digest generation	16
4	Quantum security goals of AXIS Hash	18
5	Software and hardware configuration for performance evaluation.	22
6	Benchmark message sizes.	23
7	AXIS-512 reference throughput.	24
8	AXIS-768 reference throughput.	24
9	AXIS-1024 reference throughput.	24
10	Compact FIPS202 reference performance, representative message sizes.	24
11	Reference comparison between AXIS-512 and compact FIPS202 SHA3-512.	25
12	AXIS AVX2-only optimized performance, best-of-5.	26
13	AXIS Best Available optimized performance, best-of-5.	26
14	XKCP KeccakP-1600/AVX2 single-state SHA3 performance, best-of-5.	26
15	AXIS-512 AVX2-only versus XKCP AVX2 single-state SHA3-512.	26
16	AXIS-512 Best Available versus XKCP AVX2 single-state SHA3-512.	26
17	GM/T 0005-2021 – axis512 Test Results	28
18	NIST SP 800-22 – axis512 Test Results	29
19	GM/T 0005-2021 – axis768 Test Results	38
20	NIST SP 800-22 – axis768 Test Results	39
21	GM/T 0005-2021 – axis1024 Test Results	47
22	NIST SP 800-22 – axis1024 Test Results	48

1 GLOSSARY

1.1 Terms and Acronyms

AXIS-512	The version of AXIS hash function that produces 512-bit digests.
AXIS-768	The version of AXIS hash function that produces 768-bit digests.
AXIS-1024	The version of AXIS hash function that produces 1024-bit digests.
beat	The time for executing one state update function (or one digest generation function).
digest	The output of a cryptographic hash function. Also called the hash value.
hash function	A function on bit strings in which the length of the output is fixed. The output often serves as a condensed representation of the input.
hash value	See digest.
message	A bit string of any length that is the input to the AXIS function.
register	A readable and writable (updatable) storage unit for temporarily holding a fixed-length variable. In this document, the fixed length of a register is 192.
security strength	A number associated with the amount of operations that is required to break a cryptographic algorithm or system. For a hash function, the security strength against preimage attacks is the full length of the digest, while that against collision attacks is half the digest length.
state	An array of bits that is repeatedly updated within a computational procedure. For AXIS hash functions, the state is represented as a bit-string of length $1536 = 192 \times 8$.

1.2 Notations and Conventions

ExL	An array of length 7, defined as $\text{ExL} = (11, 125, 107, 96, 32, 12, 0)$. Throughout this specification, array indices are numbered from right to left in ascending order, beginning with 0. For example, $\text{ExL}[0] = 0$, $\text{ExL}[1] = 12$, and $\text{ExL}[6] = 11$.
$\text{len}(X)$	For a bit string X (or an array X), $\text{len}(X)$ is the length of X in bits (or the number of elements stored in array X).
$m \bmod n$	For integers m and n , $m \bmod n$ is defined as the unique integer r satisfying $0 \leq r < n$ such that $m - r$ is divisible by n .
M	The input message to AXIS hash functions.
$\overline{M}$	The padded message to AXIS hash functions whose bit-length is a multiple of 64.
N_{MeBeat}	The number of beats in the message processing phase of AXIS hash functions, depending on the padded message length $\text{len}(\overline{M})$. For AXIS -512: $N_{\text{MeBeat}} = \text{len}(\overline{M})/2$. For AXIS -768: $N_{\text{MeBeat}} = \begin{cases} \frac{2}{3}\text{len}(\overline{M}), & \text{len}(\overline{M}) \bmod 192 = 0, \\ \frac{2}{3}\text{len}(\overline{M}) + \frac{64}{3}, & \text{len}(\overline{M}) \bmod 192 = 64, \\ \frac{2}{3}\text{len}(\overline{M}) + \frac{32}{3}, & \text{len}(\overline{M}) \bmod 192 = 128. \end{cases}$
	For AXIS -1024: $N_{\text{MeBeat}} = \text{len}(\overline{M})$.
N_{FinBeat}	The number of beats in the finalization phase of AXIS hash functions. For all member of AXIS , $N_{\text{FinBeat}} = 1024$.
N_{DigGen}	The number of beats in the finalization phase of AXIS hash functions. For AXIS -512: $N_{\text{DigGen}} = 256$. For AXIS -768: $N_{\text{DigGen}} = 512$. For AXIS -1024: $N_{\text{DigGen}} = 1024$.
$\mathcal{S}$	A state array is repeatedly updated within a computational procedure in AXIS hash functions, which is stored in 8 registers, denoted as $\mathcal{S} = S^7 \ S^6 \ \dots \ S^0$.
S^j	The state array stored in the j -th register, $j = 0, \dots, 7$.
UpL	An array of length 6, defined as $\text{UpL} = (188, 178, 162, 90, 75, 66)$. Similarly to ExL, $\text{UpL}[0] = 66$, $\text{UpL}[1] = 75$, and $\text{UpL}[5] = 188$.

$X[i_{k-1}, \dots, i_0]$	Concatenation of entries at positions $i_{k-1}, \dots, i_0$ of an array or a bit string X ($\text{len}(X) > i_{k-1} > \dots > i_0 \geq 0$). For example, if $X = 1110$, then $X[3, 1] = 11$ and $X[0] = 0$; if $X = [96, 32, 12, 0]$, then $X[3, 1] = [96, 12]$ and $X[0] = 0$.
$X \oplus Y$	For equal-length bit strings X and Y , $X \oplus Y$ is the bitwise XOR of X and Y . For example, $1100 \oplus 1010 = 0110$.
$X \cdot Y$	For equal-length bit strings X and Y , $X \cdot Y$ is the bitwise AND of X and Y . For example, $1100 \cdot 1010 = 1000$.
$X \parallel Y$	For strings X and Y , $X \parallel Y$ is the concatenation of X and Y . For example, $11001 \parallel 010 = 11001010$.

Note that, register superscripts are taken modulo 8, and bit indices within a register are taken modulo 192. For example, $S^{j+1} = S^0$ when $j = 7$, and $S^7[i+1] = S^7[0]$ when $i = 191$.

2 Specification of **AXIS** Hash

In this section, the **AXIS** hash functions are specified. **AXIS** is instantiated in three variants depending on the digest size, namely **AXIS-512**, **AXIS-768** and **AXIS-1024**, and produces hash outputs of 512 bits, 768 bits and 1024 bits, respectively.

2.1 State

All members of the **AXIS** hash functions operate on a state of 192-by-8 bits.

For the description and application of the state transformations, a 192-by-8 state $\mathcal{S}$ is split into eight 192-bit register words S^j , represented as

$$\mathcal{S} = S^7 \parallel S^6 \parallel \dots \parallel S^0.$$

As shown in Fig.1, the state of **AXIS** hash functions consists of 8 registers arranged in a circular structure, in which each of the three subsequent registers contributes 2 bits of nonlinear influence to the current register via the S-box mapping $\text{SBox}(\cdot)$, and the preceding register contributes 1 bit of linear influence, and concurrently, the current register absorbs 1 message bit via XOR for state update. Meanwhile, the g function also includes a nonlinear operation that extracts 6 bits from the current register and applies the S-box mapping $\text{SBox}(\cdot)$, exerting a nonlinear influence on the updated bits.

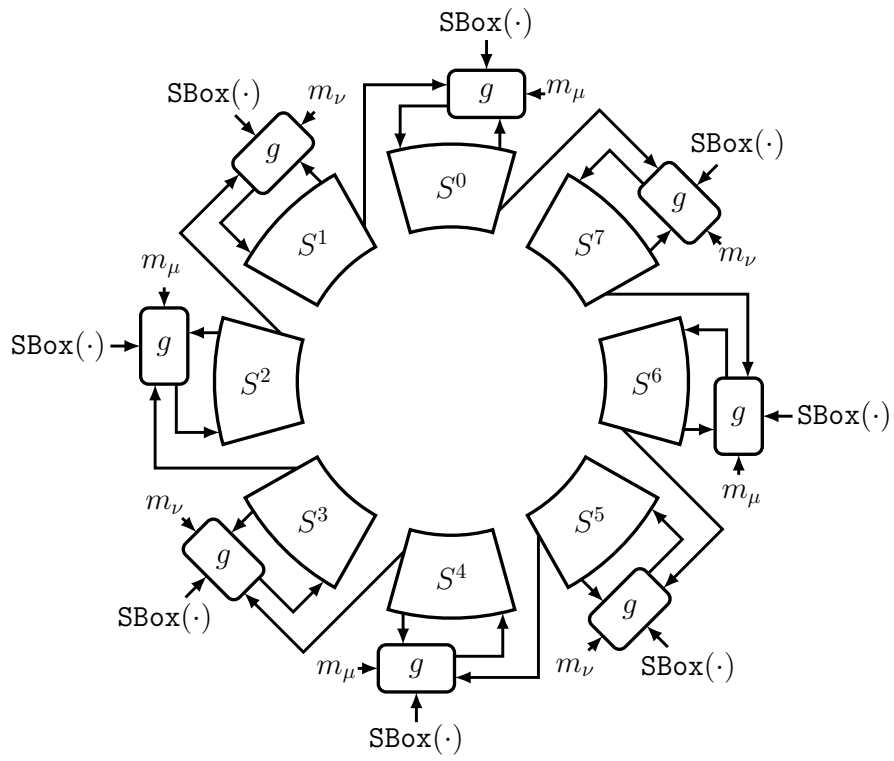


Fig. 1: The structure of **AXIS**

2.2 Mappings

The algorithms specified in this document require the use of several mappings.

Padding. The padding mapping takes a message M as input, and returns the padded message $\bar{M}$ whose length is a multiple of 64.

$\text{Pad}10^{s+1}1(M)$:

- Let $s = (-\text{len}(M) - 3) \bmod 64$.
- $\bar{M} \leftarrow M \| 1 \| 0^{s+1} \| 1 \| \text{Binary64}(\text{len}(M))$
- Return $\bar{M}$.

Here, 0^{s+1} is the string that consists of $s+1$ consecutive 0s and $\text{Binary64}(\text{len}(M))$ denotes the 64-bit binary representation of the message length $\text{len}(M)$, which is less than 2^{64} in **AXIS** hash functions.

NonLinearFunc. The non-linear function is utilized in the state update. It is 6-bit-to-1-bit Boolean function, where the nonlinear part is composed of three independent bit-product operations. The function is formalized as

$$\text{SBox}(x_5, x_4, \dots, x_0) = x_5 \cdot x_4 \oplus x_3 \cdot x_2 \oplus x_1 \cdot x_0. \quad (1)$$

Here, each product term represents a nonlinear bit multiplication over $\mathbb{F}_2$. Therefore, the differential propagation probability of an active product term is approximately 2^{-1} under the assumption of independent input differences.

Updates. Three functions, **UpRes**, **UpFull** and **ReUpFull**, are used for state updates. The first updates the state of a single register, while the latter two update the entire full state .

For **UpRes**, the register state update function takes state $S = S^7 \| S^6 \| \dots \| S^0$, a register index j , a message bit m , and two index arrays **ExL** and **UpL** as input, and returns a new state to update the j -th register.

Specifically, the elements of **ExL** are used to extract slices from the registers. A total of 7 bits are sliced: each of the three registers S^{j+5} , S^{j+3} , and S^{j+1} exerts a 2-bit nonlinear influence on the current register, while the immediately preceding register S^{j-1} exerts a 1-bit linear influence. The register update diagram is illustrated in Fig.2.

UpRes($S, j, m, \text{ExL}, \text{UpL}$) :

- $\tau \leftarrow \text{SBox}(S^{j+5}[\text{ExL}[3, 2]], S^{j+3}[\text{ExL}[5, 1]], S^{j+1}[\text{ExL}[4, 0]])$
- $b \leftarrow m \oplus \text{SBox}(S^j[\text{ExL}[5, 2, 4, 1, 3, 0]]) \oplus \tau \oplus S^{j-1}[\text{ExL}[6]],$
- $t^0 \leftarrow \bigoplus_{i \in \text{ExL}[2, 1, 0]} S^j[i] \oplus b,$
- $t^1 \leftarrow \bigoplus_{i \in \text{ExL}[5, 4, 3]} S^j[i] \oplus b,$

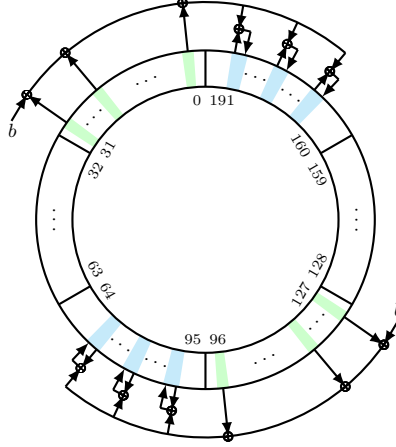


Fig. 2: The update of a register

- For $i \in \text{UpL}[2, 1, 0] : S^j[i] \leftarrow S^j[i] \oplus t^1$,
- For $i \in \text{UpL}[5, 4, 3] : S^j[i] \leftarrow S^j[i] \oplus t^0$,
- $S^j[191, 190, \dots, 0] \leftarrow S^j[0, 191, \dots, 1]$.

For **UpFull**, it takes as input the current full state $\mathcal{S}$ of **AXIS** hash functions, two bits $m_\mu \| m_\nu$ from the (padded) message stream M , and two register slice index arrays **ExL** and **UpL**, and returns the updated full state.

UpFull($\mathcal{S}, m_\mu \| m_\nu, \text{ExL}, \text{UpL}$) :

- For $j = 0$ to 7:
 - If $j \bmod 2 = 0$:
 - UpRes**($\mathcal{S}, j, m_\mu, \text{ExL}, \text{UpL}$).
 - Else:
 - UpRes**($\mathcal{S}, j, m_\nu, \text{ExL}, \text{UpL}$).

For **ReUpFull**, it takes the state $\mathcal{S}$ and slice indices, but receives 8 bits $c_7, c_6, \dots, c_0$ from a constant stream instead of the message string.

ReUpFull($\mathcal{S}, c_7 \| c_6 \| \dots \| c_0, \text{ExL}, \text{UpL}$) :

- For $j = 0$ to 7:
 - **UpRes**($\mathcal{S}, j, c_j, \text{ExL}, \text{UpL}$).

GenDigest. The digest generation functions takes the current full state $\mathcal{S}$ of **AXIS- d** , the slice index number **ExL**[0], and returns one or two hash value bits.

GenDigest($\mathcal{S}, \text{ExL}[0]$) :

- $h_\mu \leftarrow S^6[\text{ExL}[0]] \oplus S^4[\text{ExL}[0]] \oplus S^2[\text{ExL}[0]] \cdot S^0[\text{ExL}[0]],$
- $h_\nu \leftarrow S^7[\text{ExL}[0]] \oplus S^5[\text{ExL}[0]] \oplus S^3[\text{ExL}[0]] \cdot S^1[\text{ExL}[0]],$
- Return $(h_\mu \| h_\nu)[\cdot].$

2.3 Hashing

Initialization. In the **AXIS** hash function, constants for state initialization are loaded as bit strings into each register, i.e., $\text{init}S = \text{Constant7} \| \dots \| \text{Constant0}$. Table 1 lists the hexadecimal representation of the constants used in **AXIS** hash functions.

Table 1: The constants of **AXIS** Hash

Constant0	0x243F	6A88	85A3	08D3	B7E1	5162	8AED	2A6A	9E37	79B9	7F4A	7C15
Constant1	0x7160	C6B7	58B9	0A76	D81F	8650	363C	93F8	49CA	76E1	9F93	8DA4
Constant2	0x70FF	7724	ACB7	62F7	B54C	BDE3	5B65	9CC7	CD71	C257	B752	EEB5
Constant3	0x4FD6	E6B4	FBAB	7ECD	3120	355B	0549	D6B9	47EA	14CF	A7D4	3857
Constant4	0x6B63	4EF4	0491	9AAA	03F9	2D7B	7FAE	EF10	5DCD	2735	332E	4029
Constant5	0x53C1	A2C9	6C4D	E7E3	CD08	1405	6FA6	E00B	D57A	C41E	8453	48D4
Constant6	0xF3FF	9052	2F99	DAE5	352F	F663	3C6E	FBA6	ABDB	F53A	0947	6735
Constant7	0xB1A2	70D3	653F	B5B8	D945	7023	0179	F812	0D87	5B90	6F27	6D2C

For notational convenience, we discuss the algorithm in terms of 32-beat intervals. A single beat corresponds to one execution of either the state update function or the digest generation function.

Message Processing. In this phase, **AXIS-512** processes an average of 2 bits of the message stream per time beat, **AXIS-768** processes an average of 1.5 bits per time beat, and **AXIS-1024** processes an average of 1 bit per time beat.

Specifically, within per 32 beats, assuming the message stream to be processed is $\overline{M}[l-1] \| \dots \| \overline{M}[i+1] \| \overline{M}[i]$, where $l = \text{len}(\overline{M})$:

- (a) for **AXIS-512**, the bits $m_\mu \| m_\nu$ of the function **UpFull** is the lowest two consecutive bits of the pending message (i.e., $m_\mu \| m_\nu = \overline{M}[i+1] \| \overline{M}[i]$) in both even and odd beats;
- (b) for **AXIS-768**, the bits $m_\mu \| m_\nu$ of the function **UpFull** is a duplicate of the lowest bit of the pending message (i.e., $m_\mu \| m_\nu = \overline{M}[i] \| \overline{M}[i]$) in even beats, and the lowest two consecutive bits of the pending message (i.e., $m_\mu \| m_\nu = \overline{M}[i+1] \| \overline{M}[i]$) in odd beats;

- (c) for **AXIS**-1024, the bits $m_\mu \| m_\nu$ of the function **UpFull** is a duplicate of the lowest bit of the pending message (i.e., $m_\mu \| m_\nu = \overline{M}[i] \| \overline{M}[i]$) in both even and odd beats.

As a special note, for **AXIS**-768, the handling of the last two 32 beats, in the message processing period, depends on the remainder of $\text{len}(\overline{M}) \bmod 192$:

- (a) when $\text{len}(\overline{M}) \bmod 192 = 0$, the previously described rules apply;
- (b) when $\text{len}(\overline{M}) \bmod 192 = 64$, the pending message stream has length 64 and is processed with 1 bit per time beat in the last two 32 beats (i.e., $m_\mu \| m_\nu = \overline{M}[i] \| \overline{M}[i]$ in both even and odd beats);
- (c) when $\text{len}(\overline{M}) \bmod 192 = 128$, the length of the pending message stream is 128, the first 96 bits are processed with 1.5 bits per time beat as specified above, and the remaining 32 bits are processed with 1 bit per time beat (i.e., $m_\mu \| m_\nu = \overline{M}[i] \| \overline{M}[i]$ in the final 32 beats).

Finalization. In this phase, for all versions of **AXIS**, the parameter $c_7 \| c_6 \| \dots \| c_0$ of the function **ReUpFull** consists of the lowest eight consecutive bits of the pending constant stream at every beat, which equals $\dots \| \text{initS} \| \text{initS}$, determined by the length of padded message $\overline{M}$.

Digest Generation. In this phase, a register refresh operation is similar to the finalization phase. The number of beats is 256 for **AXIS**-512, 512 for **AXIS**-768, and 1024 for **AXIS**-1024. Meanwhile, the average digest output rates during this phase are 2 bits per beat for **AXIS**-512, 1.5 bits per beat for **AXIS**-768, and 1 bit per beat for **AXIS**-1024.

Specifically, within per 32 beats, for the $h_\mu \| h_\nu$ obtained from the function **GenDigest**:

- (a) for **AXIS**-512, both h_μ and h_ν contribute to the digest generation in both even and odd beats, i.e., $(h_\mu \| h_\nu)[\cdot] = h_\mu \| h_\nu$;
- (b) for **AXIS**-768, only h_μ contributes to the digest generation in even beats (i.e., $(h_\mu \| h_\nu)[\cdot] = h_\mu$), while both h_μ and h_ν contribute to the digest generation in odd beats (i.e., $(h_\mu \| h_\nu)[\cdot] = h_\mu \| h_\nu$);
- (c) for **AXIS**-1024, only h_μ contributes to the digest generation in both even and odd beats, i.e., $(h_\mu \| h_\nu)[\cdot] = h_\mu$.

3 Security Cryptanalysis

3.1 Security Goals

The security goals are given in Table 2.

Algorithm 1: The **AXIS- d** hash algorithm

Input: Message stream M , index list ExL and UpL, finalization beats N_{FinBeat} and digest generation beats N_{DigGen} .

Output: Hash Value HV .

```

// Initialization
1  $S \leftarrow \text{initS}$ 
// Message Processing
2  $\overline{M} \leftarrow \text{Pad10}^{s+1}1(64, M)$ 
3 for  $t \leftarrow 0$  to  $N_{\text{MeBeat}} - 1$  do
4    $S \leftarrow \text{UpFull}(S, m_\mu \| m_\nu, \text{ExL}, \text{UpL})$   $\triangleright m_\mu \| m_\nu$  is taken from  $\overline{M}$ 
// Finalization
5  $C \leftarrow \dots \| \text{initS} \| \text{initS}$   $\triangleright$  The bit-length of  $C$  equals  $8 \times (N_{\text{FinBeat}} + N_{\text{DigGen}})$ 
6 for  $i \leftarrow 0$  to  $N_{\text{FinBeat}} - 1$  do
7    $S \leftarrow \text{ReUpFull}(S, c_7 \| \dots \| c_0, \text{ExL}, \text{UpL})$   $\triangleright c_7 \| \dots \| c_0$  is taken from  $C$ 
// Digest Generation
8 for  $i \leftarrow 0$  to  $N_{\text{DigGen}} - 1$  do
9    $h_i \leftarrow \text{GenDigest}(S, \text{ExL})$ 
10   $S \leftarrow \text{ReUpFull}(S, c_7 \| \dots \| c_0, \text{ExL}, \text{UpL})$   $\triangleright c_7 \| \dots \| c_0$  is taken from  $C$ 
11 return  $HV = h_{N_{\text{DigGen}}-1} \| h_{N_{\text{DigGen}}-2} \| \dots \| h_0$ 

```

Table 2: Security goals of **AXIS** Hash

	Collision	Preimage	Second Preimage
AXIS-512	256-bit	512-bit	512-bit
AXIS-768	384-bit	768-bit	768-bit
AXIS-1024	512-bit	1024-bit	1024-bit

3.2 Collision Attack

Collision attacks on stream-based hash functions are difficult to realize at the message level. Instead, collision analysis is conducted by introducing controlled differences in the message and studying their propagation through the internal state. Due to the large state space, the analysis can be safely reduced to the behavior of a single register, which serves as a representative model for evaluating the collision resistance of the full state.

We analyze a stream-based hash construction with an internal state composed of eight registers:

$$\mathcal{S}_t = S_t^7 \| S_t^6 \| \dots \| S_t^0,$$

where $t \geq 0$ denotes the discrete time index of the state update function.

Due to the large state size (e.g., 1536 bits), existing automated cryptanalytic tools face significant limitations in security evaluation. Moreover, the collision resistance of the hash function can be reduced to the problem of state collisions in the underlying dynamical system.

State Update Function. We first consider a single representative register S_t^j . The update rule is defined as

$$S_{t+1}^j = \mathbf{L}S_t^j \oplus \mathbf{v} \cdot b_t, \quad (2)$$

where $\mathbf{L} \in \mathbb{F}_2^{n \times n}$ is the linear transition matrix induced by the feedback structure, $\mathbf{v} \in \mathbb{F}_2^n$ is a fixed injection vector, and $b_t \in \mathbb{F}_2$ is the nonlinear control bit at time t . The vector $\mathbf{v}$ is sparse and fixed by design, i.e., $\mathbf{v}[i] = 1$ for $i \in \text{UpL}$ and $\mathbf{v}[i] = 0$ otherwise.

Nonlinear Control Bit. At each time step t , the nonlinear control bit is defined as

$$b_t = m_t \oplus g(\cdot),$$

where m_t denotes the message-dependent input and $g(\cdot)$ is a nonlinear Boolean function defined from **UpRes**. In particular,

$$g(\cdot) = \tau \oplus \text{SBox}(S_t^j[\text{ExL}[5, 2, 4, 1, 3, 0]]) \oplus S_t^{j-1}[\text{ExL}[6]].$$

For differential analysis, all nonlinear components are abstracted into the single bit b_t .

Differential Propagation. By iterating (2), the state at time t can be written as

$$S_t^j = \mathbf{L}^t S_0^j \oplus \left(\bigoplus_{k=0}^{t-1} \mathbf{L}^{t-1-k} b_k \right) \mathbf{v}. \quad (3)$$

Let Δb_k denote the difference of b_k . The induced state difference is

$$\Delta S_t^i = \left(\bigoplus_{k=0}^{t-1} \mathbf{L}^{t-1-k} \Delta b_k \right) \mathbf{v}. \quad (4)$$

Thus, the propagation of differences is fully determined by the linear response of the system to the sequence $\{\Delta b_k\}$.

Collision Condition. A state collision at time t is achieved if

$$\Delta S_t^i = \mathbf{0}.$$

This condition can be reformulated in terms of annihilating polynomials of $\mathbf{L}$. Specifically, a state collision is achieved when the polynomial associated with the

differential sequence $\{\Delta b_k\}$ is a multiple of the minimal polynomial of $\mathbf{L}$. In this case, the accumulated linear contributions are eliminated, leading to $\Delta S_t^j = \mathbf{0}$.

Based on our analysis in the design phase, we restrict attention to a special case where the differential sequence is directly constructed from the minimal polynomial of $\mathbf{L}$.

For each candidate linear feedback configuration, the state transition matrix $\mathbf{L}$ is instantiated, and its minimal polynomial is determined. The differential sequence $\{\Delta b_k\}$ is constructed under the constraint that the induced polynomial is the minimal polynomial of $\mathbf{L}$, ensuring consistency with the annihilation condition of the linear state evolution.

Given a valid differential sequence, we model the propagation of differences using a Mixed Integer Linear Programming (MILP) framework [7]. The MILP formulation encodes the state update equations and nonlinear constraints induced by bit-product operations, and is used to track the number of active bit-product operations during differential propagation. The resulting count N_{active} is used as the objective metric for parameter selection. The selected design parameters induce a state transition matrix $\mathbf{L}$ whose minimal polynomial is given in (5).

$$\begin{aligned}
f(x) = & x^{192} + x^{121} + x^{112} + x^{105} + x^{103} + x^{101} + x^{97} + x^{96} + x^{95} + x^{94} \\
& + x^{89} + x^{87} + x^{86} + x^{85} + x^{81} + x^{80} + x^{79} + x^{78} + x^{75} + x^{74} \\
& + x^{73} + x^{72} + x^{65} + x^{64} + x^{63} + x^{61} + x^{60} + x^{58} + x^{56} + x^{55} \\
& + x^{54} + x^{53} + x^{52} + x^{50} + x^{49} + x^{46} + x^{45} + x^{44} + x^{43} \\
& + x^{41} + x^{39} + x^{37} + x^{36} + x^{34} + x^{33} + x^{31} + x^{26} + x^{25} \\
& + x^{22} + x^{20} + x^{17} + x^{16} + x^{14} + x^{10} + x^6 + x^4 + 1.
\end{aligned} \tag{5}$$

The total number of active bit-product operations obtained under this configuration is $N_{\text{active}} = 151$. After fixing the parameters, we examine the sequences generated by the update positions $\text{UpL}[0] - 1$ and $\text{UpL}[3] - 1$ over 192 consecutive time steps. The corresponding binary sequences are interpreted as polynomials over $\mathbb{F}_2[x]$, and we verify whether the resulting polynomials are coprime, i.e., whether their greatest common divisor equals 1.

We further refine the analysis by incorporating dependency effects between nonlinear operations, as observed in word-oriented constructions such as SIMON. We treat the updated bits, induced by the nonlinear feedback component, as new independent variables and construct a differential transition matrix according to the searched differential characteristics. The rank of the resulting matrix is then computed to evaluate the dimension of the constrained differential space, which determines the differential probability of the corresponding trial. Based on this analysis, the differential probability for a single register is estimated as 2^{-137} .

Security Strength Estimation. Based on the differential sequence obtained from the single-register analysis, we further evaluate the collision resistance of the full multi-register construction.

This section focuses on the evaluation methodology of the proposed security strength estimation. The goal is to identify messages and states that satisfy the differential propagation constraints. Due to the large state size, exhaustive search over message and state spaces is infeasible under current computational resources. Therefore, the collision search problem is reformulated as solving a system of differential equations, and the corresponding computational complexity is estimated to characterize the collision resistance.

We adopt a heuristic evaluation approach due to the difficulty of precisely characterizing the full state evolution. From the single-register analysis, we derive a differential sequence $\{\Delta b_k\}$ of length 193, where each Δb_k consists of contributions from message differences, nonlinear bit-product outputs, and state differences. Since eight registers interact through mutual updates, the resulting differential system cannot be explicitly expressed in closed form. However, at each time step, eight coupled differential equations are induced.

For the **AXIS**-512 variant, eight registers process two message bits per update step, while the **AXIS**-1024 variant processes one message bit per update step. The **AXIS**-768 variant adopts an interleaved one-bit/two-bit message processing mode. In the **AXIS**-512 setting, four equations corresponding to the same message bits share identical message variables. By appropriately selecting two independent message differences, two equations can be satisfied, leaving six remaining equations. For **AXIS**-1024, seven equations remain after eliminating message degrees of freedom, while **AXIS**-768 lies between these two cases.

After eliminating message differences, the remaining system is governed by state difference equations. Considering dependencies within a single register, we estimate that each register contributes approximately 137 active bit-product constraints, leading to an overall constraint scale of approximately 6×137 , 7×137 , and 6.5×137 for the three variants, respectively.

Furthermore, we assume that the message schedule provides approximately 161 degrees of freedom over 193 update steps, where the final 32 bits do not fully affect the internal state evolution. Under this assumption, for **AXIS**-1024, approximately $7 \times 137 - 161$ constraints remain to be satisfied; for **AXIS**-512, approximately $6 \times 137 - 322$; and for **AXIS**-768, approximately $6.5 \times 137 - 241.5$.

Overall, the residual constraint complexity exceeds the targeted security levels (512-bit for **AXIS**-1024, 384-bit for **AXIS**-768, and 256-bit for **AXIS**-512). This indicates that the proposed design meets the claimed security strength against collision attacks within the proposed heuristic evaluation framework.

3.3 Preimage Attack

Preimage attacks are inherently equivalent to solving systems of algebraic equations. For a hash function $H(M) = h$, given the digest h , recovering the preimage M amounts to solving a system of multivariate equations derived from the computation of H .

Specifically, let the input bits $m_0, m_1, \dots, m_{\text{len}(M)-1}$ be unknown variables, and the output bits $h_0, h_1, \dots, h_{d-1}$ be constants. Expanding the hash function into its algebraic normal form yields a system of d equations in $\text{len}(M)$ unknowns. A preimage exists iff this system admits a solution.

Reference [1] introduced a fast exhaustive search algorithm for solving a system of d Boolean equations of degree deg in n variables, under the condition $d \geq n$. Its computational cost is given by

$$T_{\text{fast}} = 4\text{deg} \cdot \log n \cdot 2^n + 2 \log n \cdot \binom{n}{\leq \text{deg}}^2 \quad (6)$$

bit operations, where the second term represents the preprocessing overhead.

Given that the number of beats in the digest generation phase exceeds the register length (192 bits) for all **AXIS** variants, it is guaranteed that all register bits contribute to the digest. We estimated the algebraic degrees of **AXIS-512** and **AXIS-1024** experimentally using the approach of [8]. The results are summarized in Table 3.

Table 3: Experimental estimation of algebraic degrees before digest generation

Target	Input size	N_{FinBeat}	Beats	Estimated degree
AXIS-512	256-bit	0	128	8
AXIS-512	256-bit	128	256	71
AXIS-512	256-bit	256	384	135
AXIS-512	512-bit	0	256	71
AXIS-512	512-bit	128	384	139
AXIS-512	512-bit	256	512	278
AXIS-1024	512-bit	0	512	≈ 292
AXIS-1024	512-bit	128	640	≈ 420
AXIS-1024	512-bit	256	768	512 (upper bound)
AXIS-1024	512-bit	512	1024	512 (upper bound)

* Beats equals $N_{\text{MeBeat}} + N_{\text{FinBeat}}$.

According to Equation (6) and the experimental estimations in Table 3, we can deduce that, **AXIS-512** achieves a security strength of 512 bits against preimage attacks. Applying the same analysis, **AXIS-768** and **AXIS-1024** both reach their respective goal security strengths against preimage attacks, as shown in Table 2.

A second-preimage attack can be interpreted as finding an alternative message that produces the same internal state evolution as a given target message. Due to the high diffusion rate and large algebraic degree of the iterative state update, the construction induces a strongly nonlinear system with deep inter-register dependencies. The existence of a known preimage does not simplify the algebraic structure of the system, as it only determines one valid trajectory without reducing the complexity of solving for alternative solutions. Consequently, we conservatively estimate that the security strength against second-preimage attacks is equivalent to the preimage security level, namely d .

3.4 Length Extension Attack

Length extension attacks are difficult to mount against stream-based hash functions. A length-extension attack is an exploit targeting hash functions built with the Merkle–Damgård construction [6,4]. Critically, the final hash output of a Merkle–Damgård hash function corresponds directly to the algorithm’s internal state after processing the last message block. This inherent property enables length extension attacks.

When a system uses $\text{Hash}(s||m_c)$ for verification, an attacker without knowledge of the secret key only needs to obtain the original message and its hash digest, as well as brute-force the length of the secret key. Leveraging the property that the hash digest equals the intermediate computation state, the attacker can reconstruct the padding bytes of the original message. The digest is then used as the starting state to append malicious data and compute a new hash value. The attacker ultimately crafts a forged message containing the padding and malicious suffix, such that the server’s calculation of $\text{Hash}(s||m_f)$ matches the counterfeit hash provided by the attacker, allowing the integrity check to be bypassed.

The output digest produced by **AXIS** constitutes merely a small subset of the hash function’s full internal state. As a result, adversaries cannot fully reconstruct the complete intermediate computation state using only the published digest. Without access to the full internal state, attackers cannot reuse an existing valid hash to compute digests for extended appended data. This structural design grants **AXIS** inherent, fundamental resistance against length extension attacks.

3.5 Quantum Attack

The classical security strength of a hash function does not directly translate to the same level of quantum security. In the absence of any structure-specific quantum attacks against the proposed construction, we evaluate the quantum security based on generic quantum attacks on hash functions.

For preimage and second-preimage attacks, Grover’s algorithm [5] provides a quadratic speedup. Therefore, an d -bit classical security level corresponds to approximately $d/2$ -bit quantum security.

For collision attacks, the BHT quantum collision attacks achieve a complexity of approximately $2^{d/3}$ queries for an ideal hash function [2]. However, for sponge-like constructions, the currently available classical-to-quantum lifting results only establish a conservative $d/4$ -bit collision-security bound [3], which is not known to be tight. Therefore, we conservatively adopt the $d/4$ bound for the quantum collision security estimation of the proposed construction.

The quantum security strength of the proposed construction is summarized in Table 4. Our quantum security evaluation is based on standard generic quantum attack models. In particular, any improvements in the generic collision complexity, e.g., from $d/4$ to $d/3$ under stronger assumptions, would translate directly to the same security level for the proposed design.

Table 4: Quantum security goals of **AXIS** Hash

	Collision	Preimage	Second Preimage
AXIS-512	128-bit	256-bit	256-bit
AXIS-768	192-bit	384-bit	384-bit
AXIS-1024	256-bit	512-bit	512-bit

4 Design Rationale

The proposed hash function is designed to support long hash outputs while maintaining an efficient and scalable update mechanism. Considering that modern cryptographic hash functions may require hundreds or even thousands of output bits, a stream-based construction is adopted. By iteratively updating a fixed-size internal state, the proposed design naturally supports variable-length message processing and different security levels.

To improve the efficiency of state processing, the internal state is composed of multiple parallel registers. Such a structure enables parallel state updates and provides scalability for different security levels. The security analysis can be initiated from a single-register model and further extended to the full multi-register construction.

For each register update, the new state of a target register is generated from both its internal feedback and external register contributions. Specifically, each register extracts linear and nonlinear feedback components from its own state, while additional information from four auxiliary registers is incorporated to update the target register. The nonlinear feedback components are generated using bit-product operations. Different extraction positions are employed for the nonlinear components to reduce the probability of cancellation among nonlinear terms and to enhance the diffusion properties. This design also provides flexibility in configuring the permutation-like state evolution induced by the blank

rounds after message processing. Therefore, the diffusion behavior can be adjusted by modifying local update parameters, which enables the construction to adapt to different security requirements.

Due to the large state space, conventional automated cryptanalysis approaches for preimage and collision resistance may become computationally infeasible within a reasonable time. Therefore, the proposed design adopts a register-based update structure that combines linear feedback and nonlinear feedback components. The state evolution of each register can be modeled as a linear transition process controlled by nonlinear feedback terms, which enables the security evaluation to be transformed into a structured algebraic analysis problem. As described in Section 3.2, the collision analysis can be converted from searching differential paths into solving differential sequences. By constructing the corresponding differential equations and evaluating their computational complexity, the collision resistance of the construction can be estimated. This analysis framework also provides a security-oriented criterion for parameter selection.

Regarding the selection of the register update mechanism, the design considers both inter-register parallelism and message-level parallelism. Since the construction is intended to support 32-bit parallel message processing, the extraction and update positions within each register are selected according to the 32-bit processing granularity, enabling efficient parallel updates.

The proposed design is optimized for hardware-friendly implementations. The state update operations mainly consist of bitwise XOR operations and bit-product operations, which can be efficiently implemented using lightweight combinational logic. The design also supports different register sizes ranging from 32 to 256 bits. With multiple registers operating independently, the construction can exploit parallel processing architectures, including SIMD-based implementations.

5 Feature Statement

The main features of the proposed hash function are summarized as follows:

- A stream-based hash construction based on multiple feedback registers, supporting variable-length outputs from a unified internal state. The register-based state design enables rapid diffusion with a relatively compact structure, while providing scalability for different hash security requirements.
- A modular multi-register architecture, where registers can be updated independently and processed in parallel, enabling efficient implementations on multi-core processors and SIMD platforms.
- A lightweight update mechanism mainly composed of bitwise XOR and bit-product operations, providing hardware-friendly implementation characteristics.
- A unified design framework that can be extended to support several cryptographic primitives, including hash functions, message authentication codes, and authenticated encryption schemes.

- A security-oriented analysis framework that reduces the evaluation of the large-state construction to a representative single-register model. The collision resistance of the full state can be heuristically estimated by analyzing the differential propagation behavior of an individual register.

6 Implementation and Test Vectors

This section presents the implementation materials and known-answer test vectors provided with the **AXIS** submission package. It is organized according to the requirements for implementation code and test-vector generation. The submitted **AXIS** package contains three hash-output instances: **AXIS-512**, **AXIS-768**, and **AXIS-1024**. The 512-bit and 1024-bit instances satisfy the mandatory output lengths, and the 768-bit instance is included as the optional intermediate output length.

6.1 Implementation Code

The implementation package is organized according to the required directory structure. Each supported algorithm instance is placed in a separate directory under both the reference and optimized implementation trees:

- Implementations/Reference_Implementation/AXIS-512
- Implementations/Reference_Implementation/AXIS-768
- Implementations/Reference_Implementation/AXIS-1024
- Implementations/Optimized_Implementation/AXIS-512
- Implementations/Optimized_Implementation/AXIS-768
- Implementations/Optimized_Implementation/AXIS-1024

An additional directory `Implementations/Additional_Implementation` is reserved for possible future implementations on other software or hardware platforms.

For clarity, the submitted implementation package follows the layout shown below:

```
Implementations/
  Reference_Implementation/
    AXIS-512/
    AXIS-768/
    AXIS-1024/
  Optimized_Implementation/
    AXIS-512/
    AXIS-768/
    AXIS-1024/
  Additional_Implementation/
  README
```

Each algorithm-instance directory contains the implementation files needed to build and test that instance, including `CryptHash_AlgorithmInstance.h/.c`, `axis_core.h/.c`, `def.h`, `KAT_CryptHash.c`, `Makefile`, and the corresponding supporting files. The exported hash interface follows the programming interface provided by the Institute of Commercial Cryptography Standards (ICCS), namely the `CryptHash(...)` interface implemented in `CryptHash_AlgorithmInstance.c`.

The interface used by each submitted instance is the ICCS hash API of the form:

```
int CryptHash(int digest_len_bits,
              const unsigned char *msg,
              unsigned long long msg_len_bits,
              unsigned char *digest);
```

The same external calling convention is used by the reference and optimized implementations, so the same ICCS driver and the same known-answer test generation flow can be used for all instances.

The reference implementation is written in ISO C and is intended to be readable and reproducible. It does not require AVX2, BMI2, or other platform-specific instructions. The optimized implementation preserves the same external API and the same bit-level outputs, while providing performance-oriented code paths for mainstream 64-bit PC processors. The optimized implementation is organized into portable optimized, AVX2 optimized, and best-available build configurations so that evaluators can reproduce both instruction-set-independent and processor-tuned measurements.

6.2 Known-Answer Test Vectors

The known-answer test vectors are provided in the `Test_Vectors` directory. As required, the test-vector package covers all submitted algorithm instances: `AXIS-512`, `AXIS-768`, and `AXIS-1024`. For each instance, the following four categories are provided:

- `KAT_2_12_[AlgorithmInstance].txt`
- `KAT_2_23_[AlgorithmInstance].txt`
- `KAT_2_33_[AlgorithmInstance].txt`
- `KAT_Loop_[AlgorithmInstance].txt`

The vectors are generated through the ICCS KAT driver and support flow included with the `API_CryptHash` package. They cover all messages from length 0 to 2^{12} bits, the dedicated 2^{23} -bit and 2^{33} -bit message cases, and the loop-test case specified for a 2^{13} -bit message.

The test-vector directory therefore has the following logical structure:

```
Test_Vectors/
  KAT_2_12_AXIS-512.txt
```

KAT_2_23_AXIS-512.txt
 KAT_2_33_AXIS-512.txt
 KAT_Loop_AXIS-512.txt
 KAT_2_12_AXIS-768.txt
 KAT_2_23_AXIS-768.txt
 KAT_2_33_AXIS-768.txt
 KAT_Loop_AXIS-768.txt
 KAT_2_12_AXIS-1024.txt
 KAT_2_23_AXIS-1024.txt
 KAT_2_33_AXIS-1024.txt
 KAT_Loop_AXIS-1024.txt

The submitted implementation code and test-vector package together provide a reproducible basis for evaluating the correctness and software performance of all three **AXIS** instances.

7 Software Performance Evaluation

This section reports the software performance of the current **AXIS** implementation on an x86-64 platform. Two levels of comparison are used. First, the **AXIS** reference implementation is compared with the readable and compact FIPS202 SHA3 reference implementation. Second, the optimized **AXIS** implementation is compared with the XKCP KeccakP-1600/AVX2 single-state SHA3 backend. The XKCP backend is from the official codebase and includes both AVX2-level optimization and assembly-level optimization, whereas the current **AXIS** AVX2 implementation uses intrinsics and does not yet include hand-written assembly optimization. The primary comparison target is **AXIS-512** versus **SHA3-512**; additional **AXIS-768**, **AXIS-1024**, **SHA3-256**, and **SHA3-384** data are included where they help show the overall performance trend.

7.1 Test Platform

Table 5: Software and hardware configuration for performance evaluation.

Item	Configuration
CPU	Intel Xeon E3-1225 v5
Operating system	Windows 10 x64
Compiler	MSYS2 GCC 15.2.0
Timing method	RDTSCL for CPU cycles and Windows QueryPerformanceCounter for elapsed time
Input data	Fixed deterministic byte-pattern buffer
Reference SHA3	XKCP readable and compact FIPS202 reference implementation
Optimized SHA3	XKCP KeccakP-1600/AVX2 single-state with FIPS202 SHA3 padding
AXIS AVX2-only	-O3 -mavx2 -mno-bmi -mno-bmi2 -mno-avx512f
AXIS Best Available	-O3 -march=native -mtune=native, allowing native BMI2 optimizations

The message sizes follow the S1–S8 classes and additionally include 1 MiB and 8 MiB messages. The optimized AXIS and XKCP AVX2 single-state results use a warmup followed by a best-of-5 measurement rule. The reference measurements are intended to compare the relative cost of reference-code implementations.

Table 6: Benchmark message sizes.

Class	Message size
S1	32 B
S2	128 B
S3	512 B
S4	1 KiB
S5	4 KiB
S6	8 KiB
S7	16 KiB
S8	64 KiB
Additional	1 MiB
Additional	8 MiB

7.2 Reference Implementation Comparison

The AXIS reference implementation does not use AVX2, BMI2, or `-march=native`. However, it still preserves algorithm-level advantages that are natural to AXIS. The internal state consists of eight 192-bit shift registers, and state transitions advance by time beats. The current reference implementation groups 32 consecutive time beats into 32-bit sequences, avoiding a fully bit-by-bit update path. For byte-aligned input, the public API also keeps a byte-aligned SoA fast path, reducing bit-level access overhead after padding.

This structure gives AXIS two advantages in the reference implementation. First, message absorption can be amortized by 32-beat block processing for long messages. Second, the finalization and digest generation phases can also advance in 32-beat blocks instead of returning to scalar state updates for every beat. Compared with the compact FIPS202 SHA3 reference implementation, the AXIS reference implementation is closer to an engineered reference implementation, while the compact FIPS202 code prioritizes readability and compactness.

As shown in Table 11, **AXIS-512** is slower than compact FIPS202 **SHA3-512** for 32 B messages. The reason is that AXIS has fixed finalization and digest generation costs, and very short messages cannot amortize these costs. Starting from 128 B, **AXIS-512** exceeds **SHA3-512**. At KiB-scale message sizes, the 32-beat block-processing advantage becomes clear, and the long-message throughput advantage is approximately 11–15x.

Table 7: AXIS-512 reference throughput.

Size	Iters	MiB/s	cyc/B	CPU cycles
32 B	20000	0.83	3819.836	2444695040
128 B	10000	3.19	991.178	1268707840
512 B	5000	9.75	323.922	829240320
1 KiB	3000	18.51	170.650	524236800
4 KiB	1000	30.46	103.698	424747008
8 KiB	500	40.13	78.701	322359296
16 KiB	200	43.71	72.254	236761907
64 KiB	80	51.19	61.706	323517153
1 MiB	4	49.05	64.393	270083817
8 MiB	1	41.76	75.635	634472366

Table 8: AXIS-768 reference throughput.

Size	Iters	MiB/s	cyc/B
32 B	20000	0.55	5718.957
128 B	10000	2.18	1447.750
512 B	5000	7.00	451.158
1 KiB	3000	10.30	306.619
4 KiB	1000	17.96	175.909
8 KiB	500	18.97	166.483
16 KiB	200	19.90	158.708
64 KiB	80	22.78	138.656
1 MiB	4	24.05	131.355
8 MiB	1	24.35	129.707

Table 9: AXIS-1024 reference throughput.

Size	Iters	MiB/s	cyc/B
32 B	20000	0.50	6313.954
128 B	10000	1.81	1745.345
512 B	5000	6.78	465.824
1 KiB	3000	11.38	277.558
4 KiB	1000	22.34	141.414
8 KiB	500	25.45	124.130
16 KiB	200	27.94	113.047
64 KiB	80	30.16	104.743
1 MiB	4	30.95	102.064
8 MiB	1	30.88	102.272

Table 10: Compact FIPS202 reference performance, representative message sizes.

Size	Iters	S3-256 MiB/s	S3-256 cyc/B	S3-384 MiB/s	S3-384 cyc/B	S3-512 MiB/s	S3-512 cyc/B
32 B	20000	1.84	1719.958	1.70	1861.005	1.71	1842.311
128 B	10000	6.85	460.853	3.62	871.791	3.04	1039.834
512 B	5000	6.04	523.143	5.51	572.725	3.08	1025.483
4 KiB	1000	6.68	473.130	4.29	736.353	3.77	838.839
64 KiB	80	7.07	446.741	5.55	569.170	3.79	833.526
1 MiB	4	7.18	440.158	4.75	664.526	3.17	996.161
8 MiB	1	7.16	441.441	5.46	578.558	3.82	826.368

Table 11: Reference comparison between AXIS-512 and compact FIPS202 SHA3-512.

Size	Iters	AXIS-512 MiB/s	SHA3-512 MiB/s	AXIS/SHA3
32 B	20000	0.83	1.71	0.49x
128 B	10000	3.19	3.04	1.05x
512 B	5000	9.75	3.08	3.17x
1 KiB	3000	18.51	3.47	5.33x
4 KiB	1000	30.46	3.77	8.08x
8 KiB	500	40.13	3.89	10.32x
16 KiB	200	43.71	3.90	11.21x
64 KiB	80	51.19	3.79	13.51x
1 MiB	4	49.05	3.17	15.47x
8 MiB	1	41.76	3.82	10.93x

7.3 Optimized Implementation Comparison

The optimized implementation keeps the same external API and bit-level output as the reference implementation, but changes the internal state layout and message-processing paths. The main optimization strategies are as follows:

- **32-way time-beat bit-slice parallelism.** The implementation uses an SoA state layout and stores the low/high 32-bit lanes of the eight registers in AVX2 vectors, advancing 32 time beats at once.
- **AVX2 state update.** AVX2 XOR, AND, shift, permute, and blend operations are used for tap extraction, SBox composition, and feedback update.
- **Byte-aligned one-shot hash path.** Byte-aligned input uses a dedicated one-shot path to avoid bit-by-bit concatenation and padded-message access in the generic streaming path.
- **AXIS-512 message-packing fast path.** A 64-bit message block is split into two 32-bit message streams. The Best Available build uses BMI2 `_pext_u64` to extract even and odd bits, while the AVX2-only build uses lookup tables to reduce the non-BMI2 extraction cost.
- **AXIS-1024 same-m fast path.** In AXIS-1024, the two input bits are equal at every beat, enabling a same-m update path and reducing message-pattern construction overhead.
- **Vector-state finalization and digest generation.** The zero-cycle and output blank-cycle phases remain in SoA/AVX2 state as much as possible, avoiding repeated conversion back to scalar state.
- **Zero-copy first streaming segment.** The first byte-aligned update may directly reference the user input buffer. The implementation falls back to an owned buffer only when later updates are appended.

For AXIS-512, the main speedups come from regular 64-bit message packing, 32-beat parallel state updates, and BMI2 bit extraction in the Best Available build. Tables 12 and 13 also show that AXIS-768 has lower optimized throughput than AXIS-512 and AXIS-1024. The main reason is that AXIS-768 has a more complex beat schedule with mixed 1-bit and 2-bit beats and special tail handling. Although AXIS-1024 absorbs only one bit per beat, its same-m rule gives it a more regular update path and good long-message throughput.

Table 12: AXIS AVX2-only optimized performance, best-of-5.

Size	Iters	A512 MiB/s	A512 cyc/B	A768 MiB/s	A768 cyc/B	A1024 MiB/s	A1024 cyc/B
32 B	120000	2.28	1383.211	1.72	1836.988	1.36	2326.339
128 B	80000	8.92	354.193	6.21	508.347	5.32	593.363
512 B	40000	33.34	94.732	17.93	176.169	19.88	158.861
1 KiB	25000	61.73	51.166	26.11	120.973	37.06	85.238
4 KiB	10000	160.07	19.732	39.63	79.703	101.28	31.186
8 KiB	6000	217.29	14.536	43.52	72.580	140.67	22.453
16 KiB	3000	278.44	11.344	43.89	71.962	162.33	19.457
64 KiB	800	299.28	10.554	48.48	65.151	223.73	14.118
1 MiB	80	329.84	9.576	52.31	60.381	252.84	12.492
8 MiB	20	401.41	7.869	52.54	60.115	252.63	12.503

Table 13: AXIS Best Available optimized performance, best-of-5.

Size	Iters	A512 MiB/s	A512 cyc/B	A768 MiB/s	A768 cyc/B	A1024 MiB/s	A1024 cyc/B
32 B	120000	2.93	1079.190	2.09	1509.410	1.72	1832.455
128 B	80000	11.49	274.976	7.57	417.518	6.78	465.644
512 B	40000	39.90	79.158	20.17	156.615	21.37	147.799
1 KiB	25000	60.66	52.071	29.42	107.349	45.90	68.816
4 KiB	10000	203.44	15.525	46.71	67.620	120.56	26.200
8 KiB	6000	294.11	10.739	51.33	61.532	166.22	19.003
16 KiB	3000	362.59	8.711	54.14	58.338	203.38	15.531
64 KiB	800	442.46	7.139	55.69	56.715	225.04	14.035
1 MiB	80	435.83	7.247	54.17	58.309	243.20	12.987
8 MiB	20	407.04	7.760	54.12	58.362	241.61	13.073

Table 14: XKCP KeccakP-1600/AVX2 single-state SHA3 performance, best-of-5.

Size	Iters	S3-256 MiB/s	S3-256 cyc/B	S3-384 MiB/s	S3-384 cyc/B	S3-512 MiB/s	S3-512 cyc/B
32 B	120000	87.28	36.189	87.12	36.253	85.50	36.942
128 B	80000	346.97	9.103	173.17	18.239	172.69	18.290
512 B	40000	339.40	9.306	278.32	11.349	172.29	18.332
1 KiB	25000	347.76	9.083	278.96	11.323	185.25	17.050
4 KiB	10000	357.77	8.828	279.05	11.319	195.30	16.173
8 KiB	6000	353.75	8.929	280.36	11.266	195.40	16.165
16 KiB	3000	361.12	8.746	281.36	11.226	196.30	16.091
64 KiB	800	365.54	8.641	279.21	11.312	194.12	16.271
1 MiB	80	359.65	8.782	278.34	11.348	194.76	16.218
8 MiB	20	359.56	8.785	276.77	11.412	192.82	16.381

Table 15: AXIS-512 AVX2-only versus XKCP AVX2 single-state SHA3-512.

Size	Iters	AXIS-512 MiB/s	SHA3-512 MiB/s	AXIS/SHA3
32 B	120000	2.28	85.50	0.03x
128 B	80000	8.92	172.69	0.05x
512 B	40000	33.34	172.29	0.19x
1 KiB	25000	61.73	185.25	0.33x
4 KiB	10000	160.07	195.30	0.82x
8 KiB	6000	217.29	195.40	1.11x
16 KiB	3000	278.44	196.30	1.42x
64 KiB	800	299.28	194.12	1.54x
1 MiB	80	329.84	194.76	1.69x
8 MiB	20	401.41	192.82	2.08x

Table 16: AXIS-512 Best Available versus XKCP AVX2 single-state SHA3-512.

Size	Iters	AXIS-512 MiB/s	SHA3-512 MiB/s	AXIS/SHA3
32 B	120000	2.93	85.50	0.03x
128 B	80000	11.49	172.69	0.07x
512 B	40000	39.90	172.29	0.23x
1 KiB	25000	60.66	185.25	0.33x
4 KiB	10000	203.44	195.30	1.04x
8 KiB	6000	294.11	195.40	1.51x
16 KiB	3000	362.59	196.30	1.85x
64 KiB	800	442.46	194.12	2.28x
1 MiB	80	435.83	194.76	2.24x
8 MiB	20	407.04	192.82	2.11x

Tables 15 and 16 show that **AXIS-512** remains significantly slower than **XKCP AVX2** single-state **SHA3-512** for short messages. This is not caused by the steady-state update throughput alone. Instead, the fixed finalization and digest generation costs dominate short messages, while the highly optimized **XKCP** single-state backend has very low startup overhead.

As the message length increases, the structural advantage of **AXIS** becomes visible. In Table 15, the **AVX2**-only build exceeds **SHA3-512** from 8 KiB onward and reaches about 2.08x at 8 MiB. In Table 16, the **Best Available** build exceeds **SHA3-512** from 4 KiB onward and reaches about 2.1–2.3x in the 64 KiB to 8 MiB range. The additional advantage of **Best Available** over **AVX2**-only mainly comes from native compilation and **BMI2 _pext_u64**, which substantially reduces the cost of splitting 64-bit message blocks into even and odd bit streams for **AXIS-512**.

7.4 Analysis and Conclusions

At the reference-implementation level, the data in Tables 7 and 11 indicate that **AXIS-512** benefits from the algorithm’s native 32-beat block progression and the byte-aligned **SoA** fast path. It outperforms compact **FIPS202 SHA3-512** reference code from 128 B onward. However, the compact **FIPS202** reference shown in Table 10 is not a high-performance **SHA3** implementation; therefore, this comparison mainly reflects reference-code complexity and natural implementation efficiency, not final optimized **SHA3** performance.

At the optimized-implementation level, Tables 15 and 16 show a clear long-message advantage over **XKCP AVX2** single-state **SHA3-512**. The reason is that **AXIS** can advance 32 time beats in parallel, and the fixed finalization and digest generation costs are amortized on long inputs. In the **Best Available** build, **BMI2** bit extraction further reduces message-packing overhead, which is consistent with the higher **AXIS-512** throughput in Table 13. Short messages remain the weak point of **AXIS** because the fixed blank-round and output-generation costs are too large relative to the input length.

As shown in Tables 8, 12, and 13, **AXIS-768** is currently the weakest among the three **AXIS** instances. Its 768-bit output instance uses a mixed beat schedule, including both 1-bit and 2-bit beats, and its tail processing depends on the padded-message length modulo 3. This schedule is less directly vectorizable than the 2-bit-per-beat rule of **AXIS-512** and the same-m 1-bit-per-beat rule of **AXIS-1024**. Further optimization may focus on **AXIS-768** message scheduling and tail-block handling.

Overall, the measurements in Tables 11, 15, and 16 support the following conclusions:

1. In the reference implementation, **AXIS-512** outperforms compact **FIPS202 SHA3-512** reference code from 128 B onward.
2. In the **AVX2**-only optimized implementation, **AXIS-512** exceeds **XKCP AVX2** single-state **SHA3-512** from 8 KiB onward.

3. In the Best Available optimized implementation, **AXIS-512** exceeds XKCP AVX2 single-state **SHA3-512** from 4 KiB onward and achieves more than 2x throughput on large messages.
4. **AXIS** remains slower than optimized **SHA3** for short messages, mainly because of fixed finalization and digest generation costs.
5. **AXIS-768** still has optimization headroom, and its mixed beat schedule is the main source of the current bottleneck.

8 Randomness Test Results for Algorithm Submission

Randomness tests were performed on three data groups to determine whether the tested data passes the GM/T 0005-2021 and NIST SP 800-22 test suites. The tested sequence length is 10^9 bits. Detailed results are listed below.

8.1 Test Results for Output Length 512 and Security Strength 256

This section reports the randomness test results for the 10^9 -bit data generated with output length 512 and security strength 256.

8.1.1 GM/T 0005-2021 Test Results

Standard	GM/T 0005-2021
Overall Result	Pass
Test File	axis512_prefix40_3B8F1A6C92 z1024.bin
Report Date	2026-06-30
Sample Count	1000
Bits per Sample	1,000,000 bits
Test Category	Standard
Report Type	GM

Table 17: GM/T 0005-2021 – axis512 Test Results

Test Item and Parameters	Parameter/Subtest	Pass Count	Q-value	Result
Frequency Test	–	987/1000	0.705466	Pass
Block Frequency Test	m=10000	991/1000	0.076658	Pass
Poker Test	m=4	990/1000	0.253122	Pass
Poker Test	m=8	989/1000	0.377007	Pass
Overlapping Template Matching Test	m=3, P1	991/1000	0.916599	Pass
Overlapping Template Matching Test	m=3, P2	994/1000	0.440975	Pass
Overlapping Template Matching Test	m=5, P1	991/1000	0.796268	Pass

Continued on next page

Test Item and Parameters	Parameter/Subtest	Pass Count	Q-value	Result
Overlapping Template Matching Test	m=5, P2	991/1000	0.286836	Pass
Runs Test	–	990/1000	0.253122	Pass
Runs Distribution Test	–	984/1000	0.851383	Pass
Longest Run Test	m=10000, zero runs	990/1000	0.026948	Pass
Longest Run Test	m=10000, one runs	990/1000	0.951205	Pass
Binary Derivation Test	m=3	993/1000	0.790621	Pass
Binary Derivation Test	m=7	993/1000	0.478839	Pass
Autocorrelation Test	m=1	990/1000	0.173770	Pass
Autocorrelation Test	m=2	992/1000	0.209948	Pass
Autocorrelation Test	m=8	988/1000	0.558502	Pass
Autocorrelation Test	m=16	988/1000	0.074791	Pass
Binary Matrix Rank Test	–	994/1000	0.026410	Pass
Cumulative Sums Test	Forward	987/1000	0.711601	Pass
Cumulative Sums Test	Backward	987/1000	0.670396	Pass
Approximate Entropy Test	m=2	992/1000	0.937919	Pass
Approximate Entropy Test	m=5	990/1000	0.933472	Pass
Linear Complexity Test	m=500	989/1000	0.334538	Pass
Linear Complexity Test	m=1000	990/1000	0.986658	Pass
Universal Statistical Test	–	988/1000	0.767582	Pass
Discrete Fourier Transform Test	–	993/1000	0.005638	Pass

8.1.2 NIST SP 800-22 Test Results

Standard	NIST SP 800-22
Overall Result	Pass
Test File	axis512_prefix40_3B8F1A6C92 z1024.bin
Report Date	2026-06-30
Sample Count	1000
Bits per Sample	1,000,000 bits
Test Category	Standard
Report Type	NIST

Table 18: NIST SP 800-22 – axis512 Test Results

Test Item and Parameters	Parameter/Subtest	Pass Count	Pt	Result
Frequency Test	–	987/1000	0.045675	Pass
Frequency Test within a Block	m=128	991/1000	0.228367	Pass
Cumulative Sums Test	–	987/1000	0.711601	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest		Pass Count	Pt	Result
Runs Test		–		990/1000	0.721777	Pass
The Longest Run of Ones in a Block		–		990/1000	0.951205	Pass
Binary Matrix Rank Test		–		994/1000	0.046568	Pass
Discrete Fourier Transform Test		–		988/1000	0.014857	Pass
Non-overlapping Matching	Template	m=9, 1/148	template	996/1000	0.939005	Pass
Non-overlapping Matching	Template	m=9, 2/148	template	989/1000	0.733899	Pass
Non-overlapping Matching	Template	m=9, 3/148	template	989/1000	0.279844	Pass
Non-overlapping Matching	Template	m=9, 4/148	template	992/1000	0.858002	Pass
Non-overlapping Matching	Template	m=9, 5/148	template	992/1000	0.680755	Pass
Non-overlapping Matching	Template	m=9, 6/148	template	989/1000	0.308561	Pass
Non-overlapping Matching	Template	m=9, 7/148	template	996/1000	0.342451	Pass
Non-overlapping Matching	Template	m=9, 8/148	template	988/1000	0.452173	Pass
Non-overlapping Matching	Template	m=9, 9/148	template	991/1000	0.568739	Pass
Non-overlapping Matching	Template	m=9, 10/148	template	990/1000	0.080027	Pass
Non-overlapping Matching	Template	m=9, 11/148	template	989/1000	0.506194	Pass
Non-overlapping Matching	Template	m=9, 12/148	template	991/1000	0.245490	Pass
Non-overlapping Matching	Template	m=9, 13/148	template	993/1000	0.528111	Pass
Non-overlapping Matching	Template	m=9, 14/148	template	994/1000	0.406499	Pass
Non-overlapping Matching	Template	m=9, 15/148	template	993/1000	0.506194	Pass
Non-overlapping Matching	Template	m=9, 16/148	template	992/1000	0.448424	Pass
Non-overlapping Matching	Template	m=9, 17/148	template	988/1000	0.263572	Pass
Non-overlapping Matching	Template	m=9, 18/148	template	994/1000	0.168112	Pass
Non-overlapping Matching	Template	m=9, 19/148	template	994/1000	0.601766	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest		Pass Count	Pt	Result
Non-overlapping Matching	Template	m=9, 20/148	template	992/1000	0.465415	Pass
Non-overlapping Matching	Template	m=9, 21/148	template	991/1000	0.576961	Pass
Non-overlapping Matching	Template	m=9, 22/148	template	984/1000	0.433590	Pass
Non-overlapping Matching	Template	m=9, 23/148	template	986/1000	0.469232	Pass
Non-overlapping Matching	Template	m=9, 24/148	template	990/1000	0.317565	Pass
Non-overlapping Matching	Template	m=9, 25/148	template	988/1000	0.476911	Pass
Non-overlapping Matching	Template	m=9, 26/148	template	990/1000	0.963497	Pass
Non-overlapping Matching	Template	m=9, 27/148	template	988/1000	0.502247	Pass
Non-overlapping Matching	Template	m=9, 28/148	template	994/1000	0.643366	Pass
Non-overlapping Matching	Template	m=9, 29/148	template	994/1000	0.916599	Pass
Non-overlapping Matching	Template	m=9, 30/148	template	987/1000	0.994005	Pass
Non-overlapping Matching	Template	m=9, 31/148	template	995/1000	0.344048	Pass
Non-overlapping Matching	Template	m=9, 32/148	template	986/1000	0.428095	Pass
Non-overlapping Matching	Template	m=9, 33/148	template	988/1000	0.591409	Pass
Non-overlapping Matching	Template	m=9, 34/148	template	991/1000	0.419021	Pass
Non-overlapping Matching	Template	m=9, 35/148	template	986/1000	0.392456	Pass
Non-overlapping Matching	Template	m=9, 36/148	template	991/1000	0.064418	Pass
Non-overlapping Matching	Template	m=9, 37/148	template	989/1000	0.653773	Pass
Non-overlapping Matching	Template	m=9, 38/148	template	989/1000	0.877083	Pass
Non-overlapping Matching	Template	m=9, 39/148	template	988/1000	0.012829	Pass
Non-overlapping Matching	Template	m=9, 40/148	template	988/1000	0.239266	Pass
Non-overlapping Matching	Template	m=9, 41/148	template	993/1000	0.873987	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest		Pass Count	Pt	Result
Non-overlapping Matching	Template	m=9, 42/148	template	992/1000	0.319084	Pass
Non-overlapping Matching	Template	m=9, 43/148	template	990/1000	0.177628	Pass
Non-overlapping Matching	Template	m=9, 44/148	template	989/1000	0.721777	Pass
Non-overlapping Matching	Template	m=9, 45/148	template	990/1000	0.193767	Pass
Non-overlapping Matching	Template	m=9, 46/148	template	986/1000	0.371941	Pass
Non-overlapping Matching	Template	m=9, 47/148	template	989/1000	0.900569	Pass
Non-overlapping Matching	Template	m=9, 48/148	template	991/1000	0.848027	Pass
Non-overlapping Matching	Template	m=9, 49/148	template	986/1000	0.896345	Pass
Non-overlapping Matching	Template	m=9, 50/148	template	993/1000	0.903338	Pass
Non-overlapping Matching	Template	m=9, 51/148	template	990/1000	0.717714	Pass
Non-overlapping Matching	Template	m=9, 52/148	template	985/1000	0.480771	Pass
Non-overlapping Matching	Template	m=9, 53/148	template	989/1000	0.401199	Pass
Non-overlapping Matching	Template	m=9, 54/148	template	990/1000	0.753844	Pass
Non-overlapping Matching	Template	m=9, 55/148	template	987/1000	0.119508	Pass
Non-overlapping Matching	Template	m=9, 56/148	template	991/1000	0.050629	Pass
Non-overlapping Matching	Template	m=9, 57/148	template	986/1000	0.759756	Pass
Non-overlapping Matching	Template	m=9, 58/148	template	989/1000	0.773405	Pass
Non-overlapping Matching	Template	m=9, 59/148	template	992/1000	0.788728	Pass
Non-overlapping Matching	Template	m=9, 60/148	template	986/1000	0.616305	Pass
Non-overlapping Matching	Template	m=9, 61/148	template	992/1000	0.254411	Pass
Non-overlapping Matching	Template	m=9, 62/148	template	994/1000	0.593478	Pass
Non-overlapping Matching	Template	m=9, 63/148	template	984/1000	0.248014	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest		Pass Count	Pt	Result
Non-overlapping Matching	Template	m=9, 64/148	template	985/1000	0.829047	Pass
Non-overlapping Matching	Template	m=9, 65/148	template	995/1000	0.753844	Pass
Non-overlapping Matching	Template	m=9, 66/148	template	990/1000	0.784927	Pass
Non-overlapping Matching	Template	m=9, 67/148	template	990/1000	0.076658	Pass
Non-overlapping Matching	Template	m=9, 68/148	template	992/1000	0.597620	Pass
Non-overlapping Matching	Template	m=9, 69/148	template	990/1000	0.684890	Pass
Non-overlapping Matching	Template	m=9, 70/148	template	989/1000	0.805569	Pass
Non-overlapping Matching	Template	m=9, 71/148	template	987/1000	0.187581	Pass
Non-overlapping Matching	Template	m=9, 72/148	template	995/1000	0.540204	Pass
Non-overlapping Matching	Template	m=9, 73/148	template	991/1000	0.221317	Pass
Non-overlapping Matching	Template	m=9, 74/148	template	993/1000	0.729870	Pass
Non-overlapping Matching	Template	m=9, 75/148	template	996/1000	0.956729	Pass
Non-overlapping Matching	Template	m=9, 76/148	template	991/1000	0.459717	Pass
Non-overlapping Matching	Template	m=9, 77/148	template	988/1000	0.678686	Pass
Non-overlapping Matching	Template	m=9, 78/148	template	992/1000	0.765632	Pass
Non-overlapping Matching	Template	m=9, 79/148	template	988/1000	0.689019	Pass
Non-overlapping Matching	Template	m=9, 80/148	template	991/1000	0.954930	Pass
Non-overlapping Matching	Template	m=9, 81/148	template	991/1000	0.246750	Pass
Non-overlapping Matching	Template	m=9, 82/148	template	989/1000	0.326749	Pass
Non-overlapping Matching	Template	m=9, 83/148	template	993/1000	0.452173	Pass
Non-overlapping Matching	Template	m=9, 84/148	template	990/1000	0.184549	Pass
Non-overlapping Matching	Template	m=9, 85/148	template	992/1000	0.001079	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest		Pass Count	Pt	Result
Non-overlapping Matching	Template	m=9, 86/148	template	990/1000	0.792508	Pass
Non-overlapping Matching	Template	m=9, 87/148	template	995/1000	0.919131	Pass
Non-overlapping Matching	Template	m=9, 88/148	template	992/1000	0.225998	Pass
Non-overlapping Matching	Template	m=9, 89/148	template	985/1000	0.062427	Pass
Non-overlapping Matching	Template	m=9, 90/148	template	991/1000	0.305599	Pass
Non-overlapping Matching	Template	m=9, 91/148	template	985/1000	0.737915	Pass
Non-overlapping Matching	Template	m=9, 92/148	template	991/1000	0.212184	Pass
Non-overlapping Matching	Template	m=9, 93/148	template	992/1000	0.167184	Pass
Non-overlapping Matching	Template	m=9, 94/148	template	995/1000	0.674543	Pass
Non-overlapping Matching	Template	m=9, 95/148	template	983/1000	0.637119	Pass
Non-overlapping Matching	Template	m=9, 96/148	template	986/1000	0.344048	Pass
Non-overlapping Matching	Template	m=9, 97/148	template	991/1000	0.765632	Pass
Non-overlapping Matching	Template	m=9, 98/148	template	992/1000	0.246750	Pass
Non-overlapping Matching	Template	m=9, 99/148	template	992/1000	0.574903	Pass
Non-overlapping Matching	Template	m=9, 100/148	template	987/1000	0.982454	Pass
Non-overlapping Matching	Template	m=9, 101/148	template	989/1000	0.230755	Pass
Non-overlapping Matching	Template	m=9, 102/148	template	988/1000	0.275709	Pass
Non-overlapping Matching	Template	m=9, 103/148	template	990/1000	0.235589	Pass
Non-overlapping Matching	Template	m=9, 104/148	template	990/1000	0.245490	Pass
Non-overlapping Matching	Template	m=9, 105/148	template	989/1000	0.908760	Pass
Non-overlapping Matching	Template	m=9, 106/148	template	993/1000	0.751866	Pass
Non-overlapping Matching	Template	m=9, 107/148	template	990/1000	0.439122	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest	Pass Count	Pt	Result
Non-overlapping Matching	Template	m=9, template 108/148	992/1000	0.830808	Pass
Non-overlapping Matching	Template	m=9, template 109/148	985/1000	0.314544	Pass
Non-overlapping Matching	Template	m=9, template 110/148	993/1000	0.878618	Pass
Non-overlapping Matching	Template	m=9, template 111/148	993/1000	0.298282	Pass
Non-overlapping Matching	Template	m=9, template 112/148	989/1000	0.821937	Pass
Non-overlapping Matching	Template	m=9, template 113/148	990/1000	0.461612	Pass
Non-overlapping Matching	Template	m=9, template 114/148	997/1000	0.620465	Pass
Non-overlapping Matching	Template	m=9, template 115/148	990/1000	0.103753	Pass
Non-overlapping Matching	Template	m=9, template 116/148	992/1000	0.943242	Pass
Non-overlapping Matching	Template	m=9, template 117/148	991/1000	0.175691	Pass
Non-overlapping Matching	Template	m=9, template 118/148	990/1000	0.645448	Pass
Non-overlapping Matching	Template	m=9, template 119/148	992/1000	0.486588	Pass
Non-overlapping Matching	Template	m=9, template 120/148	994/1000	0.747898	Pass
Non-overlapping Matching	Template	m=9, template 121/148	985/1000	0.682823	Pass
Non-overlapping Matching	Template	m=9, template 122/148	987/1000	0.049664	Pass
Non-overlapping Matching	Template	m=9, template 123/148	990/1000	0.532132	Pass
Non-overlapping Matching	Template	m=9, template 124/148	982/1000	0.313041	Pass
Non-overlapping Matching	Template	m=9, template 125/148	986/1000	0.242986	Pass
Non-overlapping Matching	Template	m=9, template 126/148	983/1000	0.486588	Pass
Non-overlapping Matching	Template	m=9, template 127/148	993/1000	0.500279	Pass
Non-overlapping Matching	Template	m=9, template 128/148	989/1000	0.471146	Pass
Non-overlapping Matching	Template	m=9, template 129/148	990/1000	0.939005	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest	Pass Count	Pt	Result
Non-overlapping Matching	Template	m=9, template 130/148	985/1000	0.078567	Pass
Non-overlapping Matching	Template	m=9, template 131/148	990/1000	0.784927	Pass
Non-overlapping Matching	Template	m=9, template 132/148	993/1000	0.344048	Pass
Non-overlapping Matching	Template	m=9, template 133/148	988/1000	0.068571	Pass
Non-overlapping Matching	Template	m=9, template 134/148	992/1000	0.587274	Pass
Non-overlapping Matching	Template	m=9, template 135/148	988/1000	0.510153	Pass
Non-overlapping Matching	Template	m=9, template 136/148	993/1000	0.649612	Pass
Non-overlapping Matching	Template	m=9, template 137/148	981/1000	0.249284	Pass
Non-overlapping Matching	Template	m=9, template 138/148	994/1000	0.440975	Pass
Non-overlapping Matching	Template	m=9, template 139/148	987/1000	0.326749	Pass
Non-overlapping Matching	Template	m=9, template 140/148	991/1000	0.313041	Pass
Non-overlapping Matching	Template	m=9, template 141/148	992/1000	0.502247	Pass
Non-overlapping Matching	Template	m=9, template 142/148	992/1000	0.067722	Pass
Non-overlapping Matching	Template	m=9, template 143/148	992/1000	0.301194	Pass
Non-overlapping Matching	Template	m=9, template 144/148	989/1000	0.556460	Pass
Non-overlapping Matching	Template	m=9, template 145/148	987/1000	0.947308	Pass
Non-overlapping Matching	Template	m=9, template 146/148	986/1000	0.556460	Pass
Non-overlapping Matching	Template	m=9, template 147/148	986/1000	0.011545	Pass
Non-overlapping Matching	Template	m=9, template 148/148	993/1000	0.757790	Pass
Overlapping Template Matching		m=9	992/1000	0.639202	Pass
Maurer's Universal Statistical Test		—	988/1000	0.818343	Pass
Approximate Entropy Test		m=10	992/1000	0.546283	Pass
Random Excursions Test		x=-4	621/628	0.498820	Pass

Continued on next page

Test Item and Parameters	Parameter/Subtest	Pass Count	Pt	Result
Random Excursions Test	x=-3	616/628	0.933179	Pass
Random Excursions Test	x=-2	624/628	0.416697	Pass
Random Excursions Test	x=-1	623/628	0.895932	Pass
Random Excursions Test	x=1	621/628	0.393858	Pass
Random Excursions Test	x=2	622/628	0.143213	Pass
Random Excursions Test	x=3	622/628	0.489351	Pass
Random Excursions Test	x=4	617/628	0.874288	Pass
Random Excursions Variant	x=-9	617/628	0.317614	Pass
Random Excursions Variant	x=-8	620/628	0.687355	Pass
Random Excursions Variant	x=-7	624/628	0.633761	Pass
Random Excursions Variant	x=-6	620/628	0.473746	Pass
Random Excursions Variant	x=-5	616/628	0.600228	Pass
Random Excursions Variant	x=-4	620/628	0.931334	Pass
Random Excursions Variant	x=-3	618/628	0.960334	Pass
Random Excursions Variant	x=-2	618/628	0.322530	Pass
Random Excursions Variant	x=-1	622/628	0.431340	Pass
Random Excursions Variant	x=1	620/628	0.293814	Pass
Random Excursions Variant	x=2	621/628	0.755946	Pass
Random Excursions Variant	x=3	624/628	0.222556	Pass
Random Excursions Variant	x=4	624/628	0.505174	Pass
Random Excursions Variant	x=5	625/628	0.273504	Pass
Random Excursions Variant	x=6	622/628	0.945329	Pass
Random Excursions Variant	x=7	622/628	0.707249	Pass
Random Excursions Variant	x=8	620/628	0.405185	Pass
Random Excursions Variant	x=9	621/628	0.966798	Pass
Serial Test	m=16, subtest 1	994/1000	0.368587	Pass
Serial Test	m=16, subtest 2	991/1000	0.347257	Pass
Linear Complexity Test	m=500	989/1000	0.334538	Pass

Conclusion: This data group passes the GM/T 0005-2021 and NIST SP 800-22 tests. The total test length is 10^9 bits. Test parameters: GM/T 0005-2021: block frequency m=10000; poker m=4/8; overlapping template m=3/5; binary derivation m=3/7; autocorrelation m=1/2/8/16; approximate entropy m=2/5; linear complexity m=500/1000; NIST SP 800-22: block frequency m=128; non-overlapping template m=9; overlapping template m=9; approximate entropy m=10; serial m=16; linear complexity m=500.

8.2 Test Results for Output Length 768 and Security Strength 384

This section reports the randomness test results for the 10^9 -bit data generated with output length 768 and security strength 384.

8.2.1 GM/T 0005-2021 Test Results

Standard	GM/T 0005-2021
Overall Result	Pass
Test File	axis768_prefix40_3B8F1A6C92 z1024.bin
Report Date	2026-06-30
Sample Count	1000
Bits per Sample	1,000,000 bits
Test Category	Standard
Report Type	GM

Table 19: GM/T 0005-2021 – axis768 Test Results

Test Item and Parameters	Parameter/Subtest	Pass Count	Q-value	Result
Frequency Test	–	991/1000	0.783019	Pass
Block Frequency Test	m=10000	989/1000	0.827279	Pass
Poker Test	m=4	987/1000	0.302657	Pass
Poker Test	m=8	988/1000	0.757790	Pass
Overlapping Template Matching Test	m=3, P1	991/1000	0.337688	Pass
Overlapping Template Matching Test	m=3, P2	989/1000	0.130369	Pass
Overlapping Template Matching Test	m=5, P1	992/1000	0.424453	Pass
Overlapping Template Matching Test	m=5, P2	990/1000	0.371941	Pass
Runs Test	–	984/1000	0.794391	Pass
Runs Distribution Test	–	988/1000	0.350485	Pass
Longest Run Test	m=10000, zero runs	993/1000	0.152902	Pass
Longest Run Test	m=10000, one runs	988/1000	0.607993	Pass
Binary Derivation Test	m=3	988/1000	0.530120	Pass
Binary Derivation Test	m=7	987/1000	0.004270	Pass
Autocorrelation Test	m=1	984/1000	0.807412	Pass
Autocorrelation Test	m=2	992/1000	0.380407	Pass
Autocorrelation Test	m=8	991/1000	0.996677	Pass
Autocorrelation Test	m=16	983/1000	0.597620	Pass
Binary Matrix Rank Test	–	990/1000	0.697257	Pass
Cumulative Sums Test	Forward	992/1000	0.699313	Pass
Cumulative Sums Test	Backward	995/1000	0.467322	Pass
Approximate Entropy Test	m=2	991/1000	0.337688	Pass
Approximate Entropy Test	m=5	990/1000	0.263572	Pass
Linear Complexity Test	m=500	989/1000	0.622546	Pass

Continued on next page

Test Item and Parameters	Parameter/Subtest	Pass Count	Q-value	Result
Linear Complexity Test	m=1000	990/1000	0.807412	Pass
Universal Statistical Test	–	994/1000	0.798139	Pass
Discrete Fourier Transform Test	–	989/1000	0.186566	Pass

8.2.2 NIST SP 800-22 Test Results

Standard	NIST SP 800-22
Overall Result	Pass
Test File	axis768_prefix40_3B8F1A6C92 z1024.bin
Report Date	2026-06-30
Sample Count	1000
Bits per Sample	1,000,000 bits
Test Category	Standard
Report Type	NIST

Table 20: NIST SP 800-22 – axis768 Test Results

Test Item and Parameters	Parameter/Subtest	Pass Count	Pt	Result
Frequency Test	–	991/1000	0.660012	Pass
Frequency Test within a Block	m=128	990/1000	0.476911	Pass
Cumulative Sums Test	–	992/1000	0.699313	Pass
Runs Test	–	984/1000	0.994005	Pass
The Longest Run of Ones in a Block	–	988/1000	0.607993	Pass
Binary Matrix Rank Test	–	990/1000	0.784927	Pass
Discrete Fourier Transform Test	–	987/1000	0.366918	Pass
Non-overlapping Matching	Template m=9, 1/148 template	988/1000	0.442831	Pass
Non-overlapping Matching	Template m=9, 2/148 template	990/1000	0.534146	Pass
Non-overlapping Matching	Template m=9, 3/148 template	990/1000	0.360287	Pass
Non-overlapping Matching	Template m=9, 4/148 template	986/1000	0.647530	Pass
Non-overlapping Matching	Template m=9, 5/148 template	993/1000	0.889118	Pass
Non-overlapping Matching	Template m=9, 6/148 template	992/1000	0.206629	Pass
Non-overlapping Matching	Template m=9, 7/148 template	989/1000	0.777265	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest		Pass Count	Pt	Result
Non-overlapping Matching	Template	m=9, 8/148	template	992/1000	0.952152	Pass
Non-overlapping Matching	Template	m=9, 9/148	template	991/1000	0.043368	Pass
Non-overlapping Matching	Template	m=9, 10/148	template	987/1000	0.182550	Pass
Non-overlapping Matching	Template	m=9, 11/148	template	992/1000	0.033362	Pass
Non-overlapping Matching	Template	m=9, 12/148	template	989/1000	0.796268	Pass
Non-overlapping Matching	Template	m=9, 13/148	template	988/1000	0.244236	Pass
Non-overlapping Matching	Template	m=9, 14/148	template	991/1000	0.566688	Pass
Non-overlapping Matching	Template	m=9, 15/148	template	983/1000	0.486588	Pass
Non-overlapping Matching	Template	m=9, 16/148	template	991/1000	0.257004	Pass
Non-overlapping Matching	Template	m=9, 17/148	template	988/1000	0.452173	Pass
Non-overlapping Matching	Template	m=9, 18/148	template	988/1000	0.496351	Pass
Non-overlapping Matching	Template	m=9, 19/148	template	988/1000	0.227180	Pass
Non-overlapping Matching	Template	m=9, 20/148	template	990/1000	0.188601	Pass
Non-overlapping Matching	Template	m=9, 21/148	template	993/1000	0.498313	Pass
Non-overlapping Matching	Template	m=9, 22/148	template	987/1000	0.689019	Pass
Non-overlapping Matching	Template	m=9, 23/148	template	985/1000	0.382115	Pass
Non-overlapping Matching	Template	m=9, 24/148	template	990/1000	0.601766	Pass
Non-overlapping Matching	Template	m=9, 25/148	template	989/1000	0.147815	Pass
Non-overlapping Matching	Template	m=9, 26/148	template	991/1000	0.743915	Pass
Non-overlapping Matching	Template	m=9, 27/148	template	987/1000	0.649612	Pass
Non-overlapping Matching	Template	m=9, 28/148	template	986/1000	0.317565	Pass
Non-overlapping Matching	Template	m=9, 29/148	template	990/1000	0.841226	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest		Pass Count	Pt	Result
Non-overlapping Matching	Template	m=9, 30/148	template	990/1000	0.267573	Pass
Non-overlapping Matching	Template	m=9, 31/148	template	989/1000	0.713641	Pass
Non-overlapping Matching	Template	m=9, 32/148	template	992/1000	0.836048	Pass
Non-overlapping Matching	Template	m=9, 33/148	template	994/1000	0.347257	Pass
Non-overlapping Matching	Template	m=9, 34/148	template	987/1000	0.116065	Pass
Non-overlapping Matching	Template	m=9, 35/148	template	990/1000	0.836048	Pass
Non-overlapping Matching	Template	m=9, 36/148	template	986/1000	0.907419	Pass
Non-overlapping Matching	Template	m=9, 37/148	template	994/1000	0.429923	Pass
Non-overlapping Matching	Template	m=9, 38/148	template	990/1000	0.761719	Pass
Non-overlapping Matching	Template	m=9, 39/148	template	988/1000	0.368587	Pass
Non-overlapping Matching	Template	m=9, 40/148	template	988/1000	0.417219	Pass
Non-overlapping Matching	Template	m=9, 41/148	template	990/1000	0.755819	Pass
Non-overlapping Matching	Template	m=9, 42/148	template	995/1000	0.305599	Pass
Non-overlapping Matching	Template	m=9, 43/148	template	991/1000	0.982454	Pass
Non-overlapping Matching	Template	m=9, 44/148	template	990/1000	0.442831	Pass
Non-overlapping Matching	Template	m=9, 45/148	template	994/1000	0.026768	Pass
Non-overlapping Matching	Template	m=9, 46/148	template	991/1000	0.492436	Pass
Non-overlapping Matching	Template	m=9, 47/148	template	990/1000	0.576961	Pass
Non-overlapping Matching	Template	m=9, 48/148	template	987/1000	0.153763	Pass
Non-overlapping Matching	Template	m=9, 49/148	template	988/1000	0.566688	Pass
Non-overlapping Matching	Template	m=9, 50/148	template	988/1000	0.703417	Pass
Non-overlapping Matching	Template	m=9, 51/148	template	991/1000	0.649612	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest		Pass Count	Pt	Result
Non-overlapping Matching	Template	m=9, 52/148	template	986/1000	0.987079	Pass
Non-overlapping Matching	Template	m=9, 53/148	template	989/1000	0.504219	Pass
Non-overlapping Matching	Template	m=9, 54/148	template	991/1000	0.385543	Pass
Non-overlapping Matching	Template	m=9, 55/148	template	990/1000	0.074791	Pass
Non-overlapping Matching	Template	m=9, 56/148	template	990/1000	0.699313	Pass
Non-overlapping Matching	Template	m=9, 57/148	template	991/1000	0.128132	Pass
Non-overlapping Matching	Template	m=9, 58/148	template	989/1000	0.068999	Pass
Non-overlapping Matching	Template	m=9, 59/148	template	986/1000	0.839507	Pass
Non-overlapping Matching	Template	m=9, 60/148	template	995/1000	0.250558	Pass
Non-overlapping Matching	Template	m=9, 61/148	template	993/1000	0.678686	Pass
Non-overlapping Matching	Template	m=9, 62/148	template	994/1000	0.358641	Pass
Non-overlapping Matching	Template	m=9, 63/148	template	993/1000	0.662091	Pass
Non-overlapping Matching	Template	m=9, 64/148	template	986/1000	0.713641	Pass
Non-overlapping Matching	Template	m=9, 65/148	template	990/1000	0.207730	Pass
Non-overlapping Matching	Template	m=9, 66/148	template	996/1000	0.056069	Pass
Non-overlapping Matching	Template	m=9, 67/148	template	993/1000	0.007369	Pass
Non-overlapping Matching	Template	m=9, 68/148	template	991/1000	0.851383	Pass
Non-overlapping Matching	Template	m=9, 69/148	template	988/1000	0.839507	Pass
Non-overlapping Matching	Template	m=9, 70/148	template	993/1000	0.370262	Pass
Non-overlapping Matching	Template	m=9, 71/148	template	990/1000	0.854708	Pass
Non-overlapping Matching	Template	m=9, 72/148	template	992/1000	0.542228	Pass
Non-overlapping Matching	Template	m=9, 73/148	template	985/1000	0.258307	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest		Pass Count	Pt	Result
Non-overlapping Matching	Template	m=9, 74/148	template	985/1000	0.129620	Pass
Non-overlapping Matching	Template	m=9, 75/148	template	988/1000	0.454053	Pass
Non-overlapping Matching	Template	m=9, 76/148	template	988/1000	0.854708	Pass
Non-overlapping Matching	Template	m=9, 77/148	template	989/1000	0.488534	Pass
Non-overlapping Matching	Template	m=9, 78/148	template	990/1000	0.827279	Pass
Non-overlapping Matching	Template	m=9, 79/148	template	993/1000	0.022605	Pass
Non-overlapping Matching	Template	m=9, 80/148	template	987/1000	0.203351	Pass
Non-overlapping Matching	Template	m=9, 81/148	template	991/1000	0.743915	Pass
Non-overlapping Matching	Template	m=9, 82/148	template	990/1000	0.209948	Pass
Non-overlapping Matching	Template	m=9, 83/148	template	989/1000	0.739918	Pass
Non-overlapping Matching	Template	m=9, 84/148	template	990/1000	0.705466	Pass
Non-overlapping Matching	Template	m=9, 85/148	template	985/1000	0.849708	Pass
Non-overlapping Matching	Template	m=9, 86/148	template	993/1000	0.011791	Pass
Non-overlapping Matching	Template	m=9, 87/148	template	989/1000	0.566688	Pass
Non-overlapping Matching	Template	m=9, 88/148	template	995/1000	0.007109	Pass
Non-overlapping Matching	Template	m=9, 89/148	template	991/1000	0.526105	Pass
Non-overlapping Matching	Template	m=9, 90/148	template	990/1000	0.502247	Pass
Non-overlapping Matching	Template	m=9, 91/148	template	988/1000	0.217857	Pass
Non-overlapping Matching	Template	m=9, 92/148	template	986/1000	0.777265	Pass
Non-overlapping Matching	Template	m=9, 93/148	template	989/1000	0.127393	Pass
Non-overlapping Matching	Template	m=9, 94/148	template	994/1000	0.610070	Pass
Non-overlapping Matching	Template	m=9, 95/148	template	992/1000	0.911413	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest		Pass Count	Pt	Result
Non-overlapping Matching	Template	m=9, 96/148	template	986/1000	0.530120	Pass
Non-overlapping Matching	Template	m=9, 97/148	template	985/1000	0.163513	Pass
Non-overlapping Matching	Template	m=9, 98/148	template	991/1000	0.628790	Pass
Non-overlapping Matching	Template	m=9, 99/148	template	988/1000	0.268917	Pass
Non-overlapping Matching	Template	m=9, 100/148	template	992/1000	0.337688	Pass
Non-overlapping Matching	Template	m=9, 101/148	template	989/1000	0.626709	Pass
Non-overlapping Matching	Template	m=9, 102/148	template	993/1000	0.832561	Pass
Non-overlapping Matching	Template	m=9, 103/148	template	991/1000	0.465415	Pass
Non-overlapping Matching	Template	m=9, 104/148	template	994/1000	0.196920	Pass
Non-overlapping Matching	Template	m=9, 105/148	template	990/1000	0.723804	Pass
Non-overlapping Matching	Template	m=9, 106/148	template	989/1000	0.120909	Pass
Non-overlapping Matching	Template	m=9, 107/148	template	987/1000	0.282626	Pass
Non-overlapping Matching	Template	m=9, 108/148	template	995/1000	0.135720	Pass
Non-overlapping Matching	Template	m=9, 109/148	template	993/1000	0.373625	Pass
Non-overlapping Matching	Template	m=9, 110/148	template	990/1000	0.856359	Pass
Non-overlapping Matching	Template	m=9, 111/148	template	991/1000	0.239266	Pass
Non-overlapping Matching	Template	m=9, 112/148	template	991/1000	0.092041	Pass
Non-overlapping Matching	Template	m=9, 113/148	template	991/1000	0.526105	Pass
Non-overlapping Matching	Template	m=9, 114/148	template	990/1000	0.745908	Pass
Non-overlapping Matching	Template	m=9, 115/148	template	992/1000	0.019721	Pass
Non-overlapping Matching	Template	m=9, 116/148	template	992/1000	0.779188	Pass
Non-overlapping Matching	Template	m=9, 117/148	template	985/1000	0.881662	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest	Pass Count	Pt	Result
Non-overlapping Matching	Template	m=9, template 118/148	986/1000	0.326749	Pass
Non-overlapping Matching	Template	m=9, template 119/148	990/1000	0.684890	Pass
Non-overlapping Matching	Template	m=9, template 120/148	993/1000	0.329850	Pass
Non-overlapping Matching	Template	m=9, template 121/148	986/1000	0.455937	Pass
Non-overlapping Matching	Template	m=9, template 122/148	991/1000	0.088226	Pass
Non-overlapping Matching	Template	m=9, template 123/148	988/1000	0.800005	Pass
Non-overlapping Matching	Template	m=9, template 124/148	993/1000	0.467322	Pass
Non-overlapping Matching	Template	m=9, template 125/148	992/1000	0.881662	Pass
Non-overlapping Matching	Template	m=9, template 126/148	991/1000	0.589341	Pass
Non-overlapping Matching	Template	m=9, template 127/148	996/1000	0.626709	Pass
Non-overlapping Matching	Template	m=9, template 128/148	990/1000	0.749884	Pass
Non-overlapping Matching	Template	m=9, template 129/148	992/1000	0.314544	Pass
Non-overlapping Matching	Template	m=9, template 130/148	988/1000	0.168112	Pass
Non-overlapping Matching	Template	m=9, template 131/148	992/1000	0.520102	Pass
Non-overlapping Matching	Template	m=9, template 132/148	985/1000	0.311542	Pass
Non-overlapping Matching	Template	m=9, template 133/148	988/1000	0.912724	Pass
Non-overlapping Matching	Template	m=9, template 134/148	994/1000	0.522100	Pass
Non-overlapping Matching	Template	m=9, template 135/148	992/1000	0.189625	Pass
Non-overlapping Matching	Template	m=9, template 136/148	990/1000	0.337688	Pass
Non-overlapping Matching	Template	m=9, template 137/148	985/1000	0.125927	Pass
Non-overlapping Matching	Template	m=9, template 138/148	989/1000	0.755819	Pass
Non-overlapping Matching	Template	m=9, template 139/148	996/1000	0.075254	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest	Pass Count	Pt	Result
Non-overlapping Matching	Template	m=9, template 140/148	989/1000	0.231956	Pass
Non-overlapping Matching	Template	m=9, template 141/148	988/1000	0.492436	Pass
Non-overlapping Matching	Template	m=9, template 142/148	993/1000	0.161703	Pass
Non-overlapping Matching	Template	m=9, template 143/148	985/1000	0.680755	Pass
Non-overlapping Matching	Template	m=9, template 144/148	995/1000	0.566688	Pass
Non-overlapping Matching	Template	m=9, template 145/148	990/1000	0.796268	Pass
Non-overlapping Matching	Template	m=9, template 146/148	994/1000	0.223648	Pass
Non-overlapping Matching	Template	m=9, template 147/148	995/1000	0.556460	Pass
Non-overlapping Matching	Template	m=9, template 148/148	985/1000	0.163513	Pass
Overlapping Template Matching		m=9	992/1000	0.711601	Pass
Maurer's Universal Statistical Test		–	994/1000	0.285427	Pass
Approximate Entropy Test		m=10	986/1000	0.461612	Pass
Random Excursions Test		x=-4	599/609	0.098036	Pass
Random Excursions Test		x=-3	597/609	0.741585	Pass
Random Excursions Test		x=-2	601/609	0.391010	Pass
Random Excursions Test		x=-1	598/609	0.962280	Pass
Random Excursions Test		x=1	605/609	0.075331	Pass
Random Excursions Test		x=2	602/609	0.512468	Pass
Random Excursions Test		x=3	602/609	0.373906	Pass
Random Excursions Test		x=4	601/609	0.758118	Pass
Random Excursions Variant		x=-9	603/609	0.790306	Pass
Random Excursions Variant		x=-8	601/609	0.445002	Pass
Random Excursions Variant		x=-7	602/609	0.435737	Pass
Random Excursions Variant		x=-6	601/609	0.721439	Pass
Random Excursions Variant		x=-5	599/609	0.414525	Pass
Random Excursions Variant		x=-4	600/609	0.090479	Pass
Random Excursions Variant		x=-3	599/609	0.758118	Pass
Random Excursions Variant		x=-2	604/609	0.871642	Pass
Random Excursions Variant		x=-1	601/609	0.088672	Pass
Random Excursions Variant		x=1	608/609	0.652733	Pass
Random Excursions Variant		x=2	602/609	0.704442	Pass
Random Excursions Variant		x=3	604/609	0.556120	Pass

Continued on next page

Test Item and Parameters	Parameter/Subtest	Pass Count	Pt	Result
Random Excursions Variant	x=4	604/609	0.986869	Pass
Random Excursions Variant	x=5	601/609	0.216143	Pass
Random Excursions Variant	x=6	600/609	0.505865	Pass
Random Excursions Variant	x=7	599/609	0.035758	Pass
Random Excursions Variant	x=8	600/609	0.552722	Pass
Random Excursions Variant	x=9	600/609	0.467004	Pass
Serial Test	m=16, subtest 1	995/1000	0.459717	Pass
Serial Test	m=16, subtest 2	992/1000	0.502247	Pass
Linear Complexity Test	m=500	989/1000	0.622546	Pass

Conclusion: This data group passes the GM/T 0005-2021 and NIST SP 800-22 tests. The total test length is 10^9 bits. Test parameters: GM/T 0005-2021: block frequency m=10000; poker m=4/8; overlapping template m=3/5; binary derivation m=3/7; autocorrelation m=1/2/8/16; approximate entropy m=2/5; linear complexity m=500/1000; NIST SP 800-22: block frequency m=128; non-overlapping template m=9; overlapping template m=9; approximate entropy m=10; serial m=16; linear complexity m=500.

8.2.3 GM/T 0005-2021 Test Results

Standard	GM/T 0005-2021
Overall Result	Pass
Test File	axis1024_prefix40_3B8F1A6C92 z1024.bin
Report Date	2026-06-30
Sample Count	1000
Bits per Sample	1,000,000 bits
Test Category	Standard
Report Type	GM

Table 21: GM/T 0005-2021 – axis1024 Test Results

Test Item and Parameters	Parameter/Subtest	Pass Count	Q-value	Result
Frequency Test	–	992/1000	0.188601	Pass
Block Frequency Test	m=10000	988/1000	0.184549	Pass
Poker Test	m=4	989/1000	0.003273	Pass
Poker Test	m=8	993/1000	0.205531	Pass
Overlapping Template Matching Test	m=3, P1	987/1000	0.474986	Pass
Overlapping Template Matching Test	m=3, P2	990/1000	0.851383	Pass
Overlapping Template Matching Test	m=5, P1	992/1000	0.220159	Pass

Continued on next page

Test Item and Parameters	Parameter/Subtest	Pass Count	Q-value	Result
Overlapping Template Matching Test	m=5, P2	987/1000	0.446556	Pass
Runs Test	–	989/1000	0.500279	Pass
Runs Distribution Test	–	986/1000	0.113372	Pass
Longest Run Test	m=10000, zero runs	994/1000	0.862883	Pass
Longest Run Test	m=10000, one runs	990/1000	0.745908	Pass
Binary Derivation Test	m=3	989/1000	0.618385	Pass
Binary Derivation Test	m=7	986/1000	0.628790	Pass
Autocorrelation Test	m=1	990/1000	0.467322	Pass
Autocorrelation Test	m=2	989/1000	0.323668	Pass
Autocorrelation Test	m=8	991/1000	0.814724	Pass
Autocorrelation Test	m=16	994/1000	0.761719	Pass
Binary Matrix Rank Test	–	985/1000	0.344048	Pass
Cumulative Sums Test	Forward	989/1000	0.814724	Pass
Cumulative Sums Test	Backward	991/1000	0.603841	Pass
Approximate Entropy Test	m=2	987/1000	0.476911	Pass
Approximate Entropy Test	m=5	991/1000	0.727851	Pass
Linear Complexity Test	m=500	991/1000	0.494392	Pass
Linear Complexity Test	m=1000	987/1000	0.021262	Pass
Universal Statistical Test	–	985/1000	0.073417	Pass
Discrete Fourier Transform Test	–	985/1000	0.753844	Pass

8.2.4 NIST SP 800-22 Test Results

Standard	NIST SP 800-22
Overall Result	Pass
Test File	axis1024_prefix40_3B8F1A6C92 z1024.bin
Report Date	2026-06-30
Sample Count	1000
Bits per Sample	1,000,000 bits
Test Category	Standard
Report Type	NIST

Table 22: NIST SP 800-22 – axis1024 Test Results

Test Item and Parameters	Parameter/Subtest	Pass Count	Pt	Result
Frequency Test	–	992/1000	0.926487	Pass
Frequency Test within a Block	m=128	986/1000	0.180568	Pass
Cumulative Sums Test	–	989/1000	0.814724	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest		Pass Count	Pt	Result
Runs Test		–		989/1000	0.512137	Pass
The Longest Run of Ones in a Block		–		990/1000	0.745908	Pass
Binary Matrix Rank Test		–		985/1000	0.257004	Pass
Discrete Fourier Transform Test		–		983/1000	0.253122	Pass
Non-overlapping Matching	Template	m=9, 1/148	template	995/1000	0.970302	Pass
Non-overlapping Matching	Template	m=9, 2/148	template	990/1000	0.292519	Pass
Non-overlapping Matching	Template	m=9, 3/148	template	993/1000	0.968128	Pass
Non-overlapping Matching	Template	m=9, 4/148	template	992/1000	0.258307	Pass
Non-overlapping Matching	Template	m=9, 5/148	template	985/1000	0.392456	Pass
Non-overlapping Matching	Template	m=9, 6/148	template	994/1000	0.554420	Pass
Non-overlapping Matching	Template	m=9, 7/148	template	988/1000	0.932333	Pass
Non-overlapping Matching	Template	m=9, 8/148	template	985/1000	0.735908	Pass
Non-overlapping Matching	Template	m=9, 9/148	template	990/1000	0.783019	Pass
Non-overlapping Matching	Template	m=9, 10/148	template	990/1000	0.145326	Pass
Non-overlapping Matching	Template	m=9, 11/148	template	989/1000	0.048716	Pass
Non-overlapping Matching	Template	m=9, 12/148	template	989/1000	0.212184	Pass
Non-overlapping Matching	Template	m=9, 13/148	template	990/1000	0.153763	Pass
Non-overlapping Matching	Template	m=9, 14/148	template	987/1000	0.779188	Pass
Non-overlapping Matching	Template	m=9, 15/148	template	989/1000	0.114712	Pass
Non-overlapping Matching	Template	m=9, 16/148	template	992/1000	0.194813	Pass
Non-overlapping Matching	Template	m=9, 17/148	template	987/1000	0.494392	Pass
Non-overlapping Matching	Template	m=9, 18/148	template	988/1000	0.058243	Pass
Non-overlapping Matching	Template	m=9, 19/148	template	993/1000	0.032923	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest		Pass Count	Pt	Result
Non-overlapping Matching	Template	m=9,	template	989/1000	0.132640	Pass
Non-overlapping Matching	Template	20/148				
Non-overlapping Matching	Template	m=9,	template	984/1000	0.079051	Pass
Non-overlapping Matching	Template	21/148				
Non-overlapping Matching	Template	m=9,	template	991/1000	0.388990	Pass
Non-overlapping Matching	Template	22/148				
Non-overlapping Matching	Template	m=9,	template	991/1000	0.190654	Pass
Non-overlapping Matching	Template	23/148				
Non-overlapping Matching	Template	m=9,	template	983/1000	0.583145	Pass
Non-overlapping Matching	Template	24/148				
Non-overlapping Matching	Template	m=9,	template	996/1000	0.729870	Pass
Non-overlapping Matching	Template	25/148				
Non-overlapping Matching	Template	m=9,	template	994/1000	0.723804	Pass
Non-overlapping Matching	Template	26/148				
Non-overlapping Matching	Template	m=9,	template	992/1000	0.947308	Pass
Non-overlapping Matching	Template	27/148				
Non-overlapping Matching	Template	m=9,	template	988/1000	0.114712	Pass
Non-overlapping Matching	Template	28/148				
Non-overlapping Matching	Template	m=9,	template	993/1000	0.378705	Pass
Non-overlapping Matching	Template	29/148				
Non-overlapping Matching	Template	m=9,	template	990/1000	0.610070	Pass
Non-overlapping Matching	Template	30/148				
Non-overlapping Matching	Template	m=9,	template	995/1000	0.185555	Pass
Non-overlapping Matching	Template	31/148				
Non-overlapping Matching	Template	m=9,	template	991/1000	0.779188	Pass
Non-overlapping Matching	Template	32/148				
Non-overlapping Matching	Template	m=9,	template	989/1000	0.591409	Pass
Non-overlapping Matching	Template	33/148				
Non-overlapping Matching	Template	m=9,	template	988/1000	0.825505	Pass
Non-overlapping Matching	Template	34/148				
Non-overlapping Matching	Template	m=9,	template	989/1000	0.196920	Pass
Non-overlapping Matching	Template	35/148				
Non-overlapping Matching	Template	m=9,	template	997/1000	0.779188	Pass
Non-overlapping Matching	Template	36/148				
Non-overlapping Matching	Template	m=9,	template	994/1000	0.743915	Pass
Non-overlapping Matching	Template	37/148				
Non-overlapping Matching	Template	m=9,	template	991/1000	0.979226	Pass
Non-overlapping Matching	Template	38/148				
Non-overlapping Matching	Template	m=9,	template	994/1000	0.279844	Pass
Non-overlapping Matching	Template	39/148				
Non-overlapping Matching	Template	m=9,	template	985/1000	0.473064	Pass
Non-overlapping Matching	Template	40/148				
Non-overlapping Matching	Template	m=9,	template	991/1000	0.313041	Pass
Non-overlapping Matching	Template	41/148				

Continued on next page

Test Item and Parameters		Parameter/Subtest		Pass Count	Pt	Result
Non-overlapping Matching	Template	m=9, 42/148	template	986/1000	0.157251	Pass
Non-overlapping Matching	Template	m=9, 43/148	template	994/1000	0.307077	Pass
Non-overlapping Matching	Template	m=9, 44/148	template	990/1000	0.966626	Pass
Non-overlapping Matching	Template	m=9, 45/148	template	993/1000	0.199045	Pass
Non-overlapping Matching	Template	m=9, 46/148	template	988/1000	0.717714	Pass
Non-overlapping Matching	Template	m=9, 47/148	template	990/1000	0.643366	Pass
Non-overlapping Matching	Template	m=9, 48/148	template	992/1000	0.060875	Pass
Non-overlapping Matching	Template	m=9, 49/148	template	997/1000	0.662091	Pass
Non-overlapping Matching	Template	m=9, 50/148	template	996/1000	0.128874	Pass
Non-overlapping Matching	Template	m=9, 51/148	template	986/1000	0.357000	Pass
Non-overlapping Matching	Template	m=9, 52/148	template	989/1000	0.191687	Pass
Non-overlapping Matching	Template	m=9, 53/148	template	992/1000	0.093157	Pass
Non-overlapping Matching	Template	m=9, 54/148	template	986/1000	0.316052	Pass
Non-overlapping Matching	Template	m=9, 55/148	template	990/1000	0.965860	Pass
Non-overlapping Matching	Template	m=9, 56/148	template	990/1000	0.217857	Pass
Non-overlapping Matching	Template	m=9, 57/148	template	990/1000	0.302657	Pass
Non-overlapping Matching	Template	m=9, 58/148	template	987/1000	0.862883	Pass
Non-overlapping Matching	Template	m=9, 59/148	template	990/1000	0.480771	Pass
Non-overlapping Matching	Template	m=9, 60/148	template	993/1000	0.197981	Pass
Non-overlapping Matching	Template	m=9, 61/148	template	991/1000	0.979788	Pass
Non-overlapping Matching	Template	m=9, 62/148	template	993/1000	0.566688	Pass
Non-overlapping Matching	Template	m=9, 63/148	template	990/1000	0.336111	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest		Pass Count	Pt	Result
Non-overlapping Matching	Template	m=9, 64/148	template	995/1000	0.439122	Pass
Non-overlapping Matching	Template	m=9, 65/148	template	985/1000	0.199045	Pass
Non-overlapping Matching	Template	m=9, 66/148	template	990/1000	0.465415	Pass
Non-overlapping Matching	Template	m=9, 67/148	template	993/1000	0.960198	Pass
Non-overlapping Matching	Template	m=9, 68/148	template	996/1000	0.422638	Pass
Non-overlapping Matching	Template	m=9, 69/148	template	989/1000	0.450297	Pass
Non-overlapping Matching	Template	m=9, 70/148	template	993/1000	0.935716	Pass
Non-overlapping Matching	Template	m=9, 71/148	template	988/1000	0.417219	Pass
Non-overlapping Matching	Template	m=9, 72/148	template	993/1000	0.454053	Pass
Non-overlapping Matching	Template	m=9, 73/148	template	993/1000	0.084551	Pass
Non-overlapping Matching	Template	m=9, 74/148	template	992/1000	0.790621	Pass
Non-overlapping Matching	Template	m=9, 75/148	template	995/1000	0.969588	Pass
Non-overlapping Matching	Template	m=9, 76/148	template	991/1000	0.591409	Pass
Non-overlapping Matching	Template	m=9, 77/148	template	987/1000	0.123038	Pass
Non-overlapping Matching	Template	m=9, 78/148	template	992/1000	0.395940	Pass
Non-overlapping Matching	Template	m=9, 79/148	template	991/1000	0.075719	Pass
Non-overlapping Matching	Template	m=9, 80/148	template	988/1000	0.674543	Pass
Non-overlapping Matching	Template	m=9, 81/148	template	987/1000	0.452173	Pass
Non-overlapping Matching	Template	m=9, 82/148	template	989/1000	0.387264	Pass
Non-overlapping Matching	Template	m=9, 83/148	template	987/1000	0.924076	Pass
Non-overlapping Matching	Template	m=9, 84/148	template	995/1000	0.392456	Pass
Non-overlapping Matching	Template	m=9, 85/148	template	986/1000	0.078567	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest		Pass Count	Pt	Result
Non-overlapping Matching	Template	m=9, 86/148	template	987/1000	0.075719	Pass
Non-overlapping Matching	Template	m=9, 87/148	template	988/1000	0.240501	Pass
Non-overlapping Matching	Template	m=9, 88/148	template	991/1000	0.068145	Pass
Non-overlapping Matching	Template	m=9, 89/148	template	987/1000	0.524101	Pass
Non-overlapping Matching	Template	m=9, 90/148	template	989/1000	0.181557	Pass
Non-overlapping Matching	Template	m=9, 91/148	template	995/1000	0.131122	Pass
Non-overlapping Matching	Template	m=9, 92/148	template	990/1000	0.912724	Pass
Non-overlapping Matching	Template	m=9, 93/148	template	993/1000	0.530120	Pass
Non-overlapping Matching	Template	m=9, 94/148	template	988/1000	0.105618	Pass
Non-overlapping Matching	Template	m=9, 95/148	template	994/1000	0.761719	Pass
Non-overlapping Matching	Template	m=9, 96/148	template	995/1000	0.089301	Pass
Non-overlapping Matching	Template	m=9, 97/148	template	986/1000	0.052275	Pass
Non-overlapping Matching	Template	m=9, 98/148	template	990/1000	0.506194	Pass
Non-overlapping Matching	Template	m=9, 99/148	template	995/1000	0.680755	Pass
Non-overlapping Matching	Template	m=9, 100/148	template	992/1000	0.307077	Pass
Non-overlapping Matching	Template	m=9, 101/148	template	987/1000	0.568739	Pass
Non-overlapping Matching	Template	m=9, 102/148	template	988/1000	0.933472	Pass
Non-overlapping Matching	Template	m=9, 103/148	template	981/1000	0.023386	Pass
Non-overlapping Matching	Template	m=9, 104/148	template	991/1000	0.159020	Pass
Non-overlapping Matching	Template	m=9, 105/148	template	985/1000	0.739918	Pass
Non-overlapping Matching	Template	m=9, 106/148	template	987/1000	0.062427	Pass
Non-overlapping Matching	Template	m=9, 107/148	template	991/1000	0.437274	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest	Pass Count	Pt	Result
Non-overlapping Matching	Template	m=9, template 108/148	987/1000	0.807412	Pass
Non-overlapping Matching	Template	m=9, template 109/148	990/1000	0.392456	Pass
Non-overlapping Matching	Template	m=9, template 110/148	988/1000	0.450297	Pass
Non-overlapping Matching	Template	m=9, template 111/148	989/1000	0.435430	Pass
Non-overlapping Matching	Template	m=9, template 112/148	990/1000	0.992084	Pass
Non-overlapping Matching	Template	m=9, template 113/148	993/1000	0.711601	Pass
Non-overlapping Matching	Template	m=9, template 114/148	991/1000	0.055361	Pass
Non-overlapping Matching	Template	m=9, template 115/148	991/1000	0.091487	Pass
Non-overlapping Matching	Template	m=9, template 116/148	989/1000	0.361938	Pass
Non-overlapping Matching	Template	m=9, template 117/148	988/1000	0.906069	Pass
Non-overlapping Matching	Template	m=9, template 118/148	995/1000	0.064418	Pass
Non-overlapping Matching	Template	m=9, template 119/148	993/1000	0.759756	Pass
Non-overlapping Matching	Template	m=9, template 120/148	989/1000	0.973055	Pass
Non-overlapping Matching	Template	m=9, template 121/148	989/1000	0.139655	Pass
Non-overlapping Matching	Template	m=9, template 122/148	984/1000	0.072066	Pass
Non-overlapping Matching	Template	m=9, template 123/148	991/1000	0.450297	Pass
Non-overlapping Matching	Template	m=9, template 124/148	993/1000	0.626709	Pass
Non-overlapping Matching	Template	m=9, template 125/148	991/1000	0.161703	Pass
Non-overlapping Matching	Template	m=9, template 126/148	987/1000	0.725829	Pass
Non-overlapping Matching	Template	m=9, template 127/148	994/1000	0.697257	Pass
Non-overlapping Matching	Template	m=9, template 128/148	987/1000	0.548314	Pass
Non-overlapping Matching	Template	m=9, template 129/148	988/1000	0.437274	Pass

Continued on next page

Test Item and Parameters		Parameter/Subtest	Pass Count	Pt	Result
Non-overlapping Matching	Template	m=9, template 130/148	989/1000	0.969588	Pass
Non-overlapping Matching	Template	m=9, template 131/148	989/1000	0.271619	Pass
Non-overlapping Matching	Template	m=9, template 132/148	987/1000	0.069863	Pass
Non-overlapping Matching	Template	m=9, template 133/148	988/1000	0.017425	Pass
Non-overlapping Matching	Template	m=9, template 134/148	991/1000	0.695200	Pass
Non-overlapping Matching	Template	m=9, template 135/148	995/1000	0.751866	Pass
Non-overlapping Matching	Template	m=9, template 136/148	991/1000	0.832561	Pass
Non-overlapping Matching	Template	m=9, template 137/148	993/1000	0.006107	Pass
Non-overlapping Matching	Template	m=9, template 138/148	986/1000	0.761719	Pass
Non-overlapping Matching	Template	m=9, template 139/148	989/1000	0.899171	Pass
Non-overlapping Matching	Template	m=9, template 140/148	991/1000	0.048404	Pass
Non-overlapping Matching	Template	m=9, template 141/148	991/1000	0.662091	Pass
Non-overlapping Matching	Template	m=9, template 142/148	994/1000	0.595549	Pass
Non-overlapping Matching	Template	m=9, template 143/148	989/1000	0.951205	Pass
Non-overlapping Matching	Template	m=9, template 144/148	984/1000	0.083526	Pass
Non-overlapping Matching	Template	m=9, template 145/148	986/1000	0.461612	Pass
Non-overlapping Matching	Template	m=9, template 146/148	987/1000	0.524101	Pass
Non-overlapping Matching	Template	m=9, template 147/148	994/1000	0.723804	Pass
Non-overlapping Matching	Template	m=9, template 148/148	992/1000	0.834308	Pass
Overlapping Template Matching		m=9	994/1000	0.538182	Pass
Maurer's Universal Statistical Test		—	985/1000	0.512137	Pass
Approximate Entropy Test		m=10	990/1000	0.796268	Pass
Random Excursions Test		x=-4	587/596	0.829853	Pass

Continued on next page

Test Item and Parameters	Parameter/Subtest	Pass Count	Pt	Result
Random Excursions Test	x=-3	593/596	0.191336	Pass
Random Excursions Test	x=-2	595/596	0.037608	Pass
Random Excursions Test	x=-1	594/596	0.200260	Pass
Random Excursions Test	x=1	590/596	0.074307	Pass
Random Excursions Test	x=2	592/596	0.743306	Pass
Random Excursions Test	x=3	590/596	0.470757	Pass
Random Excursions Test	x=4	587/596	0.719402	Pass
Random Excursions Variant	x=-9	589/596	0.245640	Pass
Random Excursions Variant	x=-8	591/596	0.411113	Pass
Random Excursions Variant	x=-7	588/596	0.890210	Pass
Random Excursions Variant	x=-6	592/596	0.638884	Pass
Random Excursions Variant	x=-5	589/596	0.702096	Pass
Random Excursions Variant	x=-4	588/596	0.645942	Pass
Random Excursions Variant	x=-3	593/596	0.047405	Pass
Random Excursions Variant	x=-2	592/596	0.642413	Pass
Random Excursions Variant	x=-1	592/596	0.237225	Pass
Random Excursions Variant	x=1	593/596	0.151436	Pass
Random Excursions Variant	x=2	593/596	0.231060	Pass
Random Excursions Variant	x=3	595/596	0.977772	Pass
Random Excursions Variant	x=4	592/596	0.835783	Pass
Random Excursions Variant	x=5	593/596	0.596602	Pass
Random Excursions Variant	x=6	594/596	0.904593	Pass
Random Excursions Variant	x=7	592/596	0.513787	Pass
Random Excursions Variant	x=8	595/596	0.789531	Pass
Random Excursions Variant	x=9	591/596	0.861429	Pass
Serial Test	m=16, subtest 1	987/1000	0.784927	Pass
Serial Test	m=16, subtest 2	991/1000	0.816537	Pass
Linear Complexity Test	m=500	991/1000	0.494392	Pass

Conclusion: This data group passes the GM/T 0005-2021 and NIST SP 800-22 tests. The total test length is 10^9 bits. Test parameters: GM/T 0005-2021: block frequency $m=10000$; poker $m=4/8$; overlapping template $m=3/5$; binary derivation $m=3/7$; autocorrelation $m=1/2/8/16$; approximate entropy $m=2/5$; linear complexity $m=500/1000$; NIST SP 800-22: block frequency $m=128$; non-overlapping template $m=9$; overlapping template $m=9$; approximate entropy $m=10$; serial $m=16$; linear complexity $m=500$.

References

1. Bouillaguet, C., Chen, H.C., Cheng, C.M., Chou, T., Niederhagen, R., Shamir, A., Yang, B.Y.: Fast exhaustive search for polynomial systems in $\mathbb{F}_2$. In: Mangard, S., Standaert, F.X. (eds.) *Cryptographic Hardware and Embedded Systems, CHES 2010*. pp. 203–218. Springer Berlin Heidelberg, Berlin, Heidelberg (2010)
2. Brassard, G., Høyer, P., Tapp, A.: Quantum cryptanalysis of hash and claw-free functions. In: *LATIN 2002: Theoretical Informatics*. pp. 163–169 (2002)
3. Cojocaru, A., Hhan, M., Liu, Q., Yamakawa, T., Yun, A.: Quantum lifting for invertible permutations and ideal ciphers. In: *Advances in Cryptology – EUROCRYPT 2024*. Springer (2024)
4. Damgård, I.: A design principle for hash functions. In: *CRYPTO. Lecture Notes in Computer Science*, vol. 435, pp. 416–427. Springer (1989)
5. Grover, L.K.: A fast quantum mechanical algorithm for database search. *Proceedings of the 28th Annual ACM Symposium on Theory of Computing* pp. 212–219 (1996)
6. Merkle, R.C.: One way hash functions and DES. In: *CRYPTO. Lecture Notes in Computer Science*, vol. 435, pp. 428–446. Springer (1989)
7. Sun, S., Hu, L., Wang, P., Qiao, K., Ma, X., Song, L.: Automatic security evaluation and (related-key) differential characteristic search: Application to simon, present, lblock, des(l) and other bit-oriented block ciphers. In: *Advances in Cryptology – ASIACRYPT 2014*. pp. 158–178. Springer (2014)
8. Zhang, M., Sun, Y.: A new practical cube attack via recovering numerous super-polys. *IACR Cryptol. ePrint Arch.* **2025**, 2209 (2025), <https://eprint.iacr.org/2025/2209>